



O'REILLY®

AI Memory Management in Agentic Systems

By Richmond Alake

Empowering AI agents with robust memory

Meet your Instructor

Richmond Alake

- **AI Engineer & Computer Vision Specialist:** He has a solid academic foundation in computer vision, robotics, and machine learning and has been involved in software development and ML engineering roles since around 2016.
- **Technical Writer & Educator:** Richmond has authored over 200 technical articles, garnering more than 1 million views. These articles appear across platforms like Medium, MongoDB Developer Hub, and the NVIDIA Developer Blog
- **Developer Advocate:** He currently serves as a Developer Advocate in AI/ML at MongoDB, where he helps guide developers on using AI technologies effectively.
- **AI Stack Engineer:** Creator of MemoRizz and MongoDB GenAI Showcase



Course schedule

Module 1: Introduction To Agent Memory

1. **Define AI Memory:** Understand what AI memory is and how it differs from human memory, including its role in creating intelligence that mimics human cognitive abilities.
2. **Relate Human Memory to AI Systems:** Identify key types of human memory (sensory, short-term, working, long-term) and their AI analogues, understanding how humans retain, recall, and reuse information over time.
3. **Understand Agent Memory:** Learn how persistent memory in AI agents allows them to accumulate knowledge, maintain context, and adapt based on past interactions.
4. **Identify Forms of Memory in AI Agents**
Explore the various forms of AI memory—explicit, implicit, episodic, semantic—and their applications in agentic systems.
5. **Grasp Context Engineering, Memory Engineering:** Discover how managing and structuring context impacts AI performance and decision-making.
6. **Learn the Principles of Memory Engineering & Management**
Understand the design, storage, retrieval, and lifecycle management of AI memory to improve system efficiency and accuracy.

Course schedule

Module 2: Overview Information Retrieval and RAG

1. **Define RAG & When to Use:** What Retrieval-Augmented Generation is, where it beats raw LLMs, and the business impact (higher accuracy, citations, fewer hallucinations, cost control).
2. **Retrieval Modes—Lexical, Vector & Hybrid:** Strengths/limits of BM25 vs semantic vectors; when to combine them; simple fusion with **RRF** or weighted scoring.
3. **Embeddings, Distance & Index Choice:** How embeddings represent text; pick **cosine/dot/L2** (and normalize when needed); choose **HNSW / IVF / Flat** based on latency, scale, and RAM; changing metrics requires reindexing.
4. **Rerankers for Precision:** Use cross-encoders to rescore Top-K and surface truly relevant chunks; practical sizing: retrieve **50–200**, keep **5–20**; batch/cache to manage latency.
5. **Memory & Chunking Engineering:** Episodic/semantic/procedural memory in agents; chunking best practices (~**512 tokens**, **10–20%** overlap), preserve structure, enrich metadata; manage drift, PII, and retention.

Course schedule

Module 3: Hands On With Memory Engineering

1. **Key Definitions:** Understanding of key terms and phrases used in AI Memory and Agent memory management such as memory provider, memory layer, memory type and memory unit
2. **Implementation:** Step by step implementation guide of implementing processes in RAG and simple Agentic Systems
3. **Memory management with Libraries:** Step by step implementation of memory management in Agentic systems with LangMem, LangGraph, MemoryBank, MemGPT and others

Course schedule

Module 4: Deep Dive in Agent Memory Engineering and Design Patterns

1. **MemoRizz:** Introduction to the experimental but highly educational open source library MemoRizz
2. **Overview of MemoRizz:** Overview of the MemoRizz library, folder structure, key implementation considerations such as memory provider class and MemAgent class.
3. **Memory Type Implementation:** Walkthrough each memory type covered in this course with their implementation consideration, retrieval approach and memory unit construction and observation.
4. **Memory Type Utilization in AI Agents**

Module 1: AI Memory

- Overview of AI memory
- What is AI Memory
- AI Agents and Agentic Systems
- Agent Memory
- Forms of Memory in AI Agents
- Context Engineering
- Memory Engineering
- Memory Management



Why Memory

Overview of AI Memory

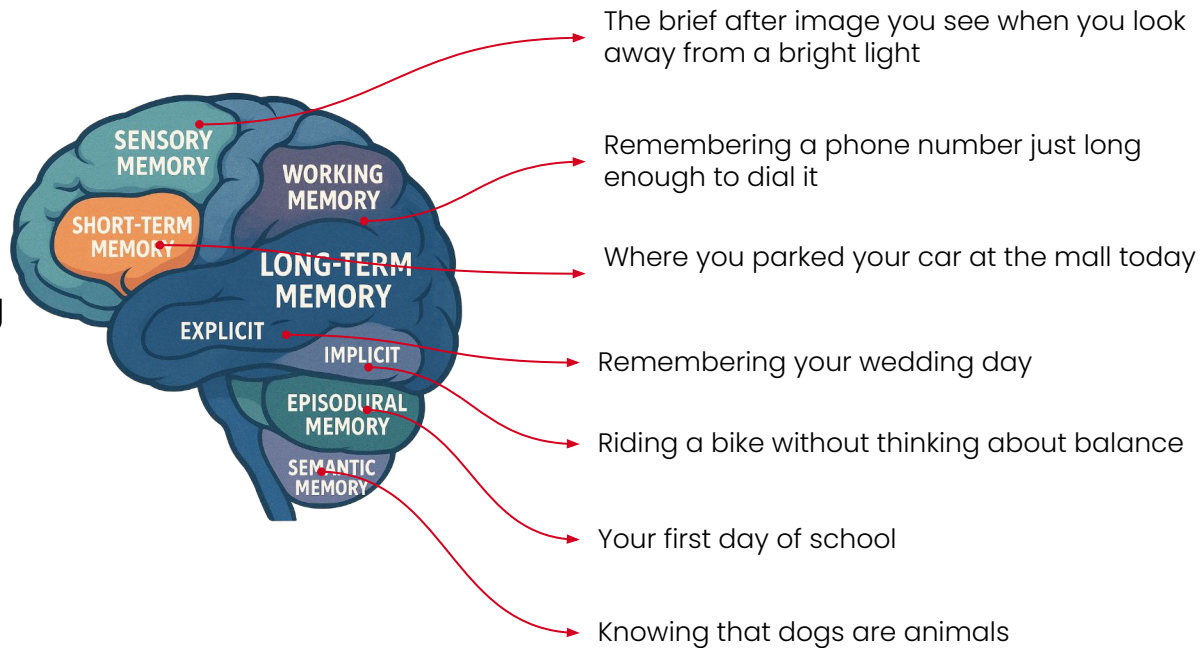
Artificial Intelligence is the scientific endeavor to create a computational form of **intelligence distinct from organic intelligence**, one that convincingly **mimics human cognitive abilities**.

Artificial General Intelligence (AGI) refers to artificial intelligence that **surpasses human performance across most tasks** traditionally considered solvable by human intelligence.

How Memory Works in Human (Loosely)

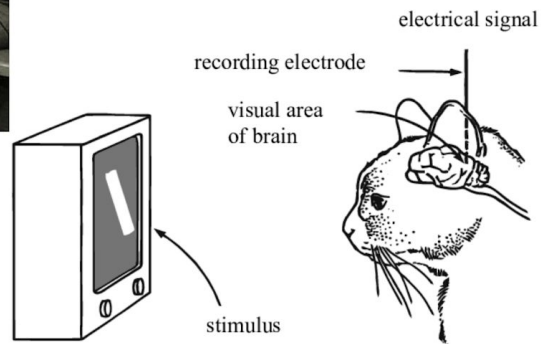
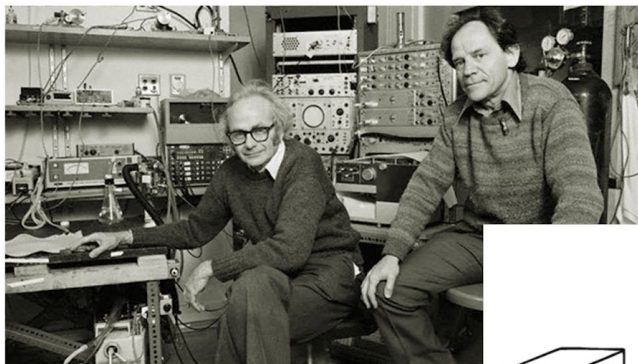
Overview of AI Memory

Humans have a good ability to **retain, recall** and **reuse** information over **short and long period** of time.



Taking Inspiration

Overview of AI Memory



What Is AI Memory?

Overview of AI Memory

AI Memory (*in Agentic Context*) == **AI Agent Memory**



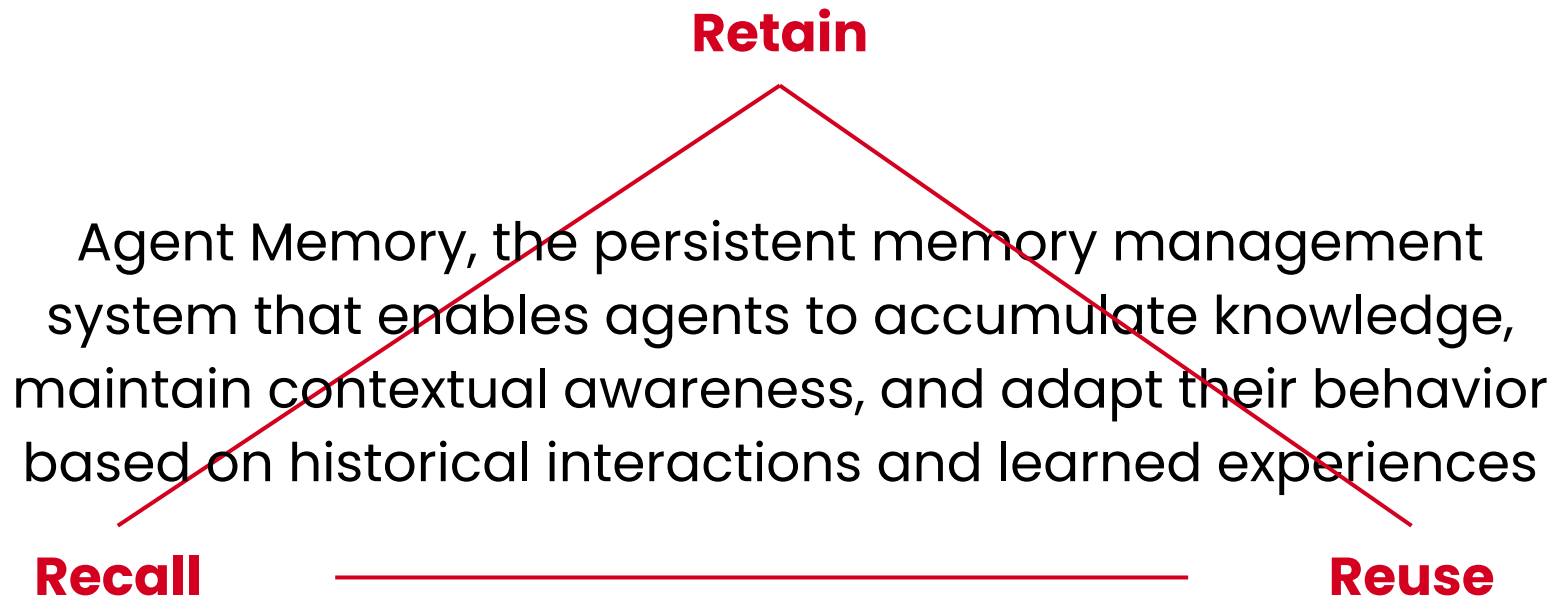
AI Memory == Agent Memory

Overview of AI Memory

Agent Memory, the persistent memory management system that enables agents to accumulate knowledge, maintain contextual awareness, and adapt their behavior based on historical interactions and learned experiences

AI Memory == Agent Memory

Overview of AI Memory



AI Memory == Agent Memory

Overview of AI Memory

Agent Memory, the ^{Retain}~~persistent~~ memory management system that enables agents to accumulate knowledge, maintain contextual awareness, and adapt their behavior based on historical interactions and learned experiences

AI Memory == Agent Memory

Overview of AI Memory

Agent Memory, the ^{Retain}~~persistent~~ memory management system that enables agents to accumulate knowledge, maintain contextual awareness, and ^{Reuse}~~adapt their behavior~~ based on historical interactions and learned experiences

AI Memory == Agent Memory

Overview of AI Memory

Agent Memory, the ^{Retain}~~persistent~~ memory management system that enables agents to accumulate knowledge, maintain contextual awareness, and ^{Reuse}~~adapt their behavior~~ based on ^{Recall}~~historical interactions and learned experiences~~

AI Memory == Agent Memory

Overview of AI Memory

Agent Memory, the ^{Retain} ~~persistent~~ ^{Reuse} ~~memory management~~ ^{Recall} ~~system~~ that enables agents to accumulate knowledge, maintain contextual awareness, and ~~adapt their behavior~~ based on ~~historical interactions and learned experiences~~

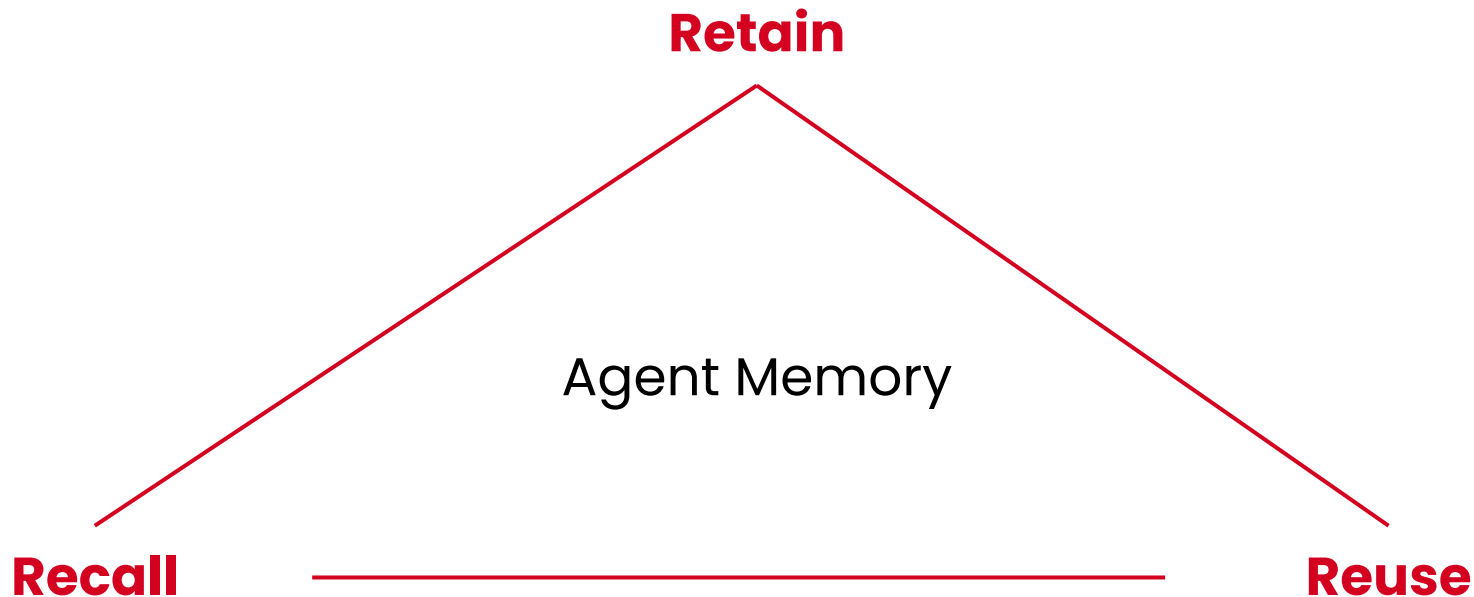
AI Memory == Agent Memory

Overview of AI Memory

Agent Memory, the ^{Retain} persistent ^{Update} ~~memory management~~ ^{Reuse} ~~system~~ that enables agents to accumulate knowledge, maintain contextual awareness, and ~~adapt their behavior~~ based on ^{Recall} ~~historical interactions and learned experiences~~

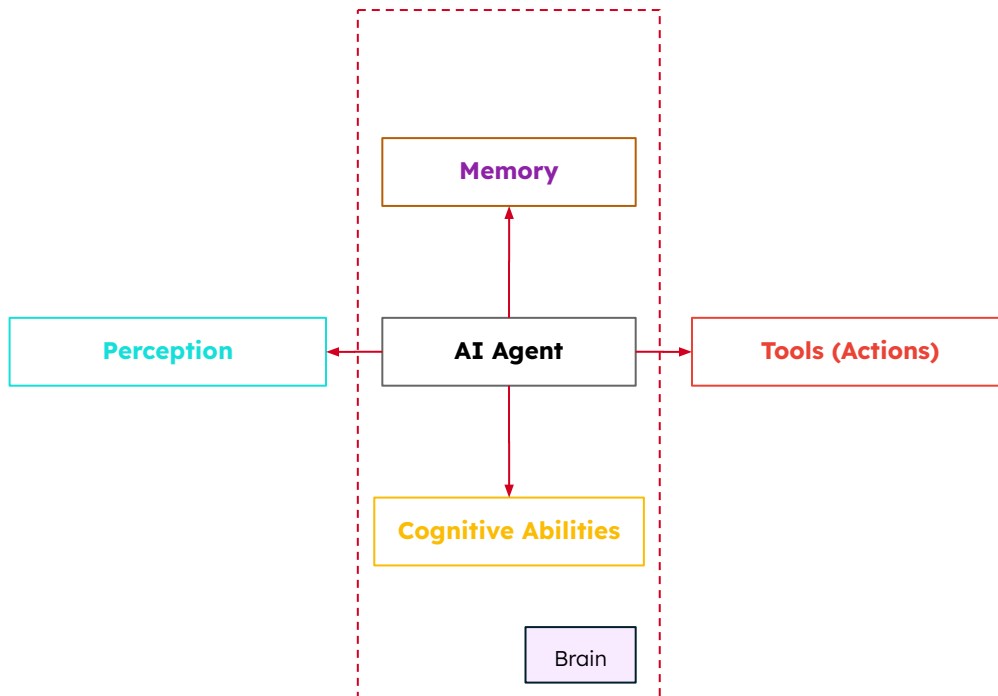
AI Memory == Agent Memory

Overview of AI Memory



What Is An AI Agent?

Overview of AI Memory



What Is An AI Agent?

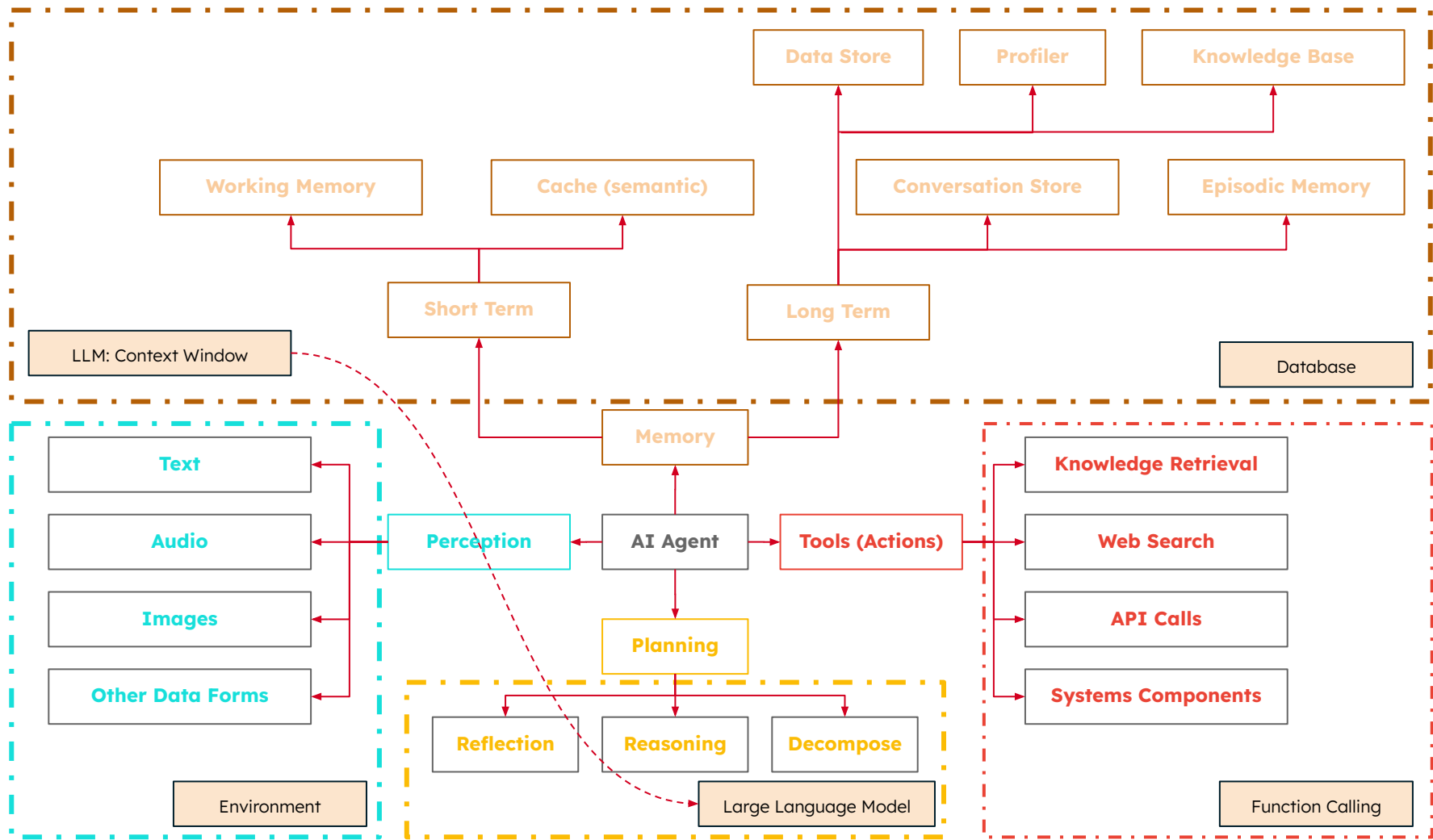
Overview of AI Memory

An AI Agent is an autonomous computational entity equipped with four essential faculties: **cognitive reasoning** powered by large language models, environmental **perception** through multimodal inputs, **actionable intelligence** via tool integration, and—most critically—**persistent memory** that enables continuous learning and adaptation.

What Is An AI Agent?

Overview of AI Memory

AI Agent is an autonomous computational entity equipped with four essential faculties: **Cognitive Abilities**, **reasoning** powered by large language models, environmental **Perception** through multimodal inputs, **actionable intelligence** via **Tools (Actions)** and—most critically—**persistent** **Memory** that enables continuous learning and adaptation.



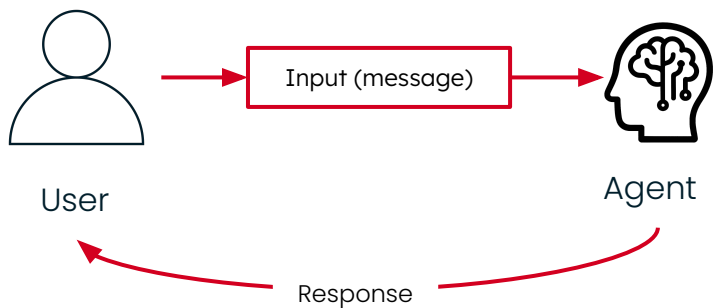
What Is An Agentic System?

Overview of AI Memory

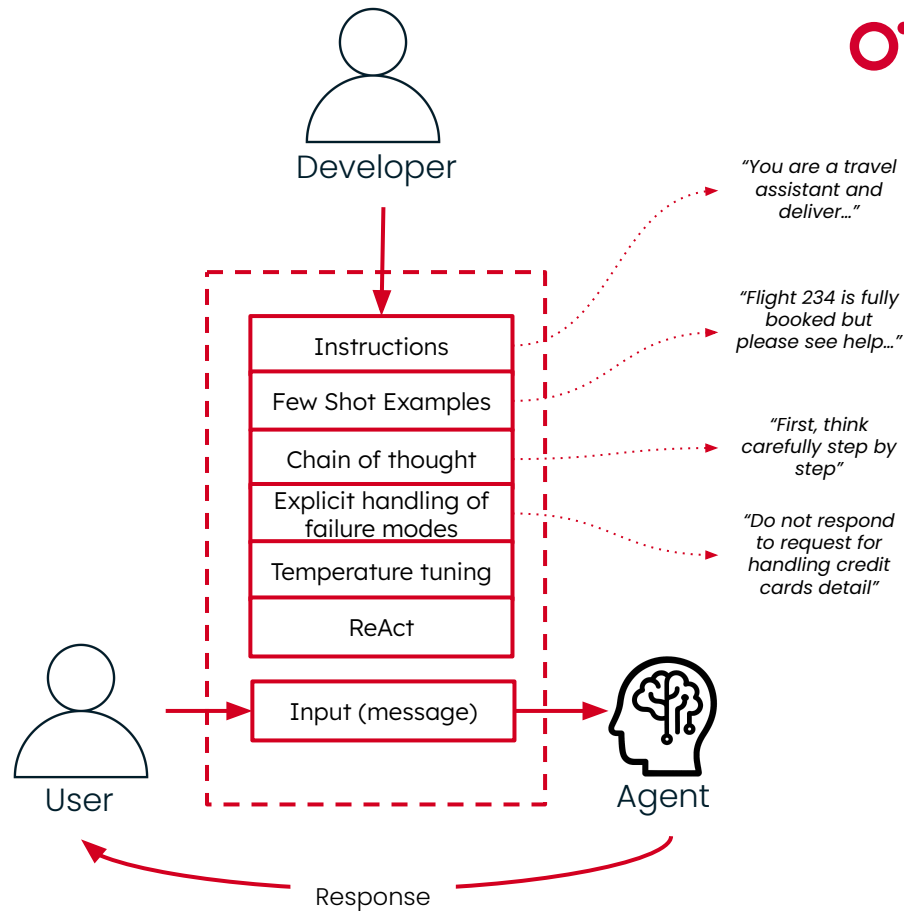
An Agentic System is a software **architecture** that orchestrates autonomous, goal-oriented behavior through the coordination of **one or more AI agents**, cognitive infrastructure, and persistent memory systems. These agents have dynamic **access to system resources**, computational capabilities, and shared infrastructure components, enabling them to achieve complex objectives.

Prompt Engineering

Overview of AI Memory



Before

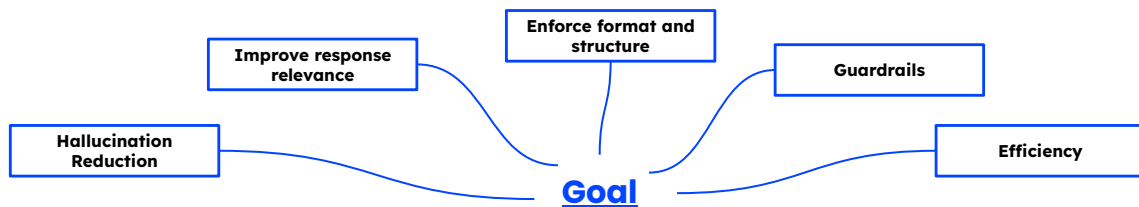


After

Prompt Engineering

Overview of AI Memory

Prompt engineering is the process of **designing and refining the combination of instructions, system messages, and inputs**—collectively called prompts—that are passed to a large language model (LLM) to ensure its outputs achieve a desired outcome.



Control and steer the behaviour of the model towards expected output, desired outcomes and overall safety.

Prompt engineering ensure that the output of the LLM's probabilistic text generation is correlated to the user's intent stated in the prompt.

Static / State Knowledge

Context Loss

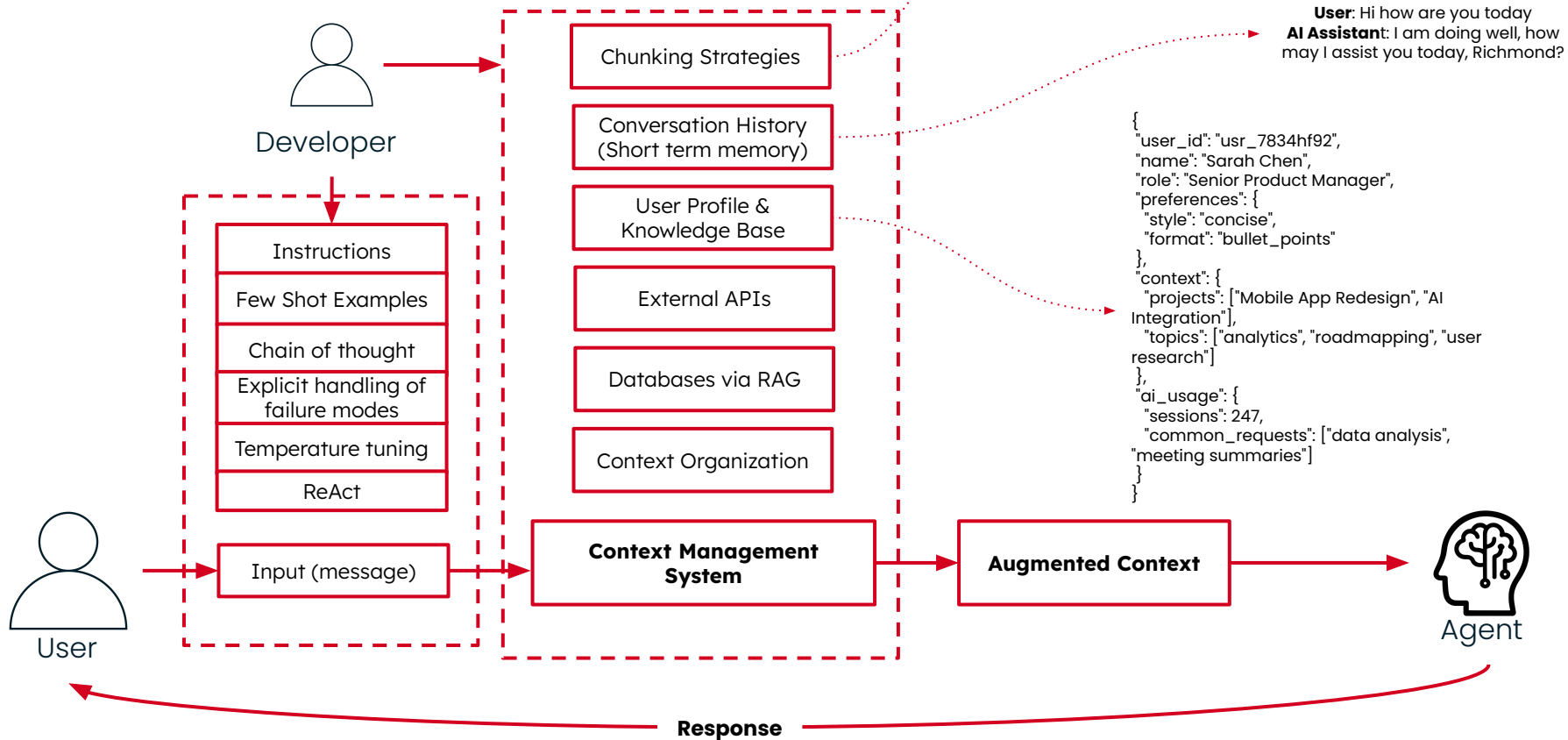
Limitations

Truncation

Phrase Sensitivity

Prompt engineering is an inherently brittle and static methodology. Its highly sensitive to the exact phrasing and tone, demanding tedious trial-and-error adjustments to guide the model. It struggles to manage long conversations or documents without significant context loss and often relies on extensive, convoluted prompts to get a specific result.

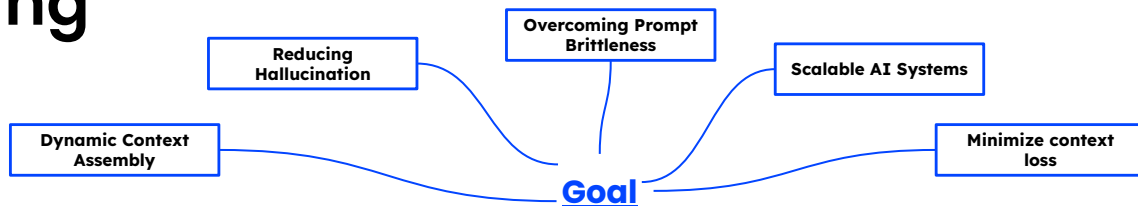
Context Engineering



Context Engineering

Overview of AI Memory

Context Engineering is the systematic practice of **dynamically constructing and managing the information** fed into a large language models context window to optimize inference outcomes.



Context engineering aims to build sophisticated and reliable AI applications and agents by dynamically assembling relevant and rich context right before LLM inference and also through selective context retrieval.

By grounding the model in real-time data via Retrieval-Augmented Generation (RAG), it significantly improves factual accuracy and reduces hallucinations.

Context Rot

Retrieval inaccuracies

Limitations

Memory drift

No True Learning or Adaptation

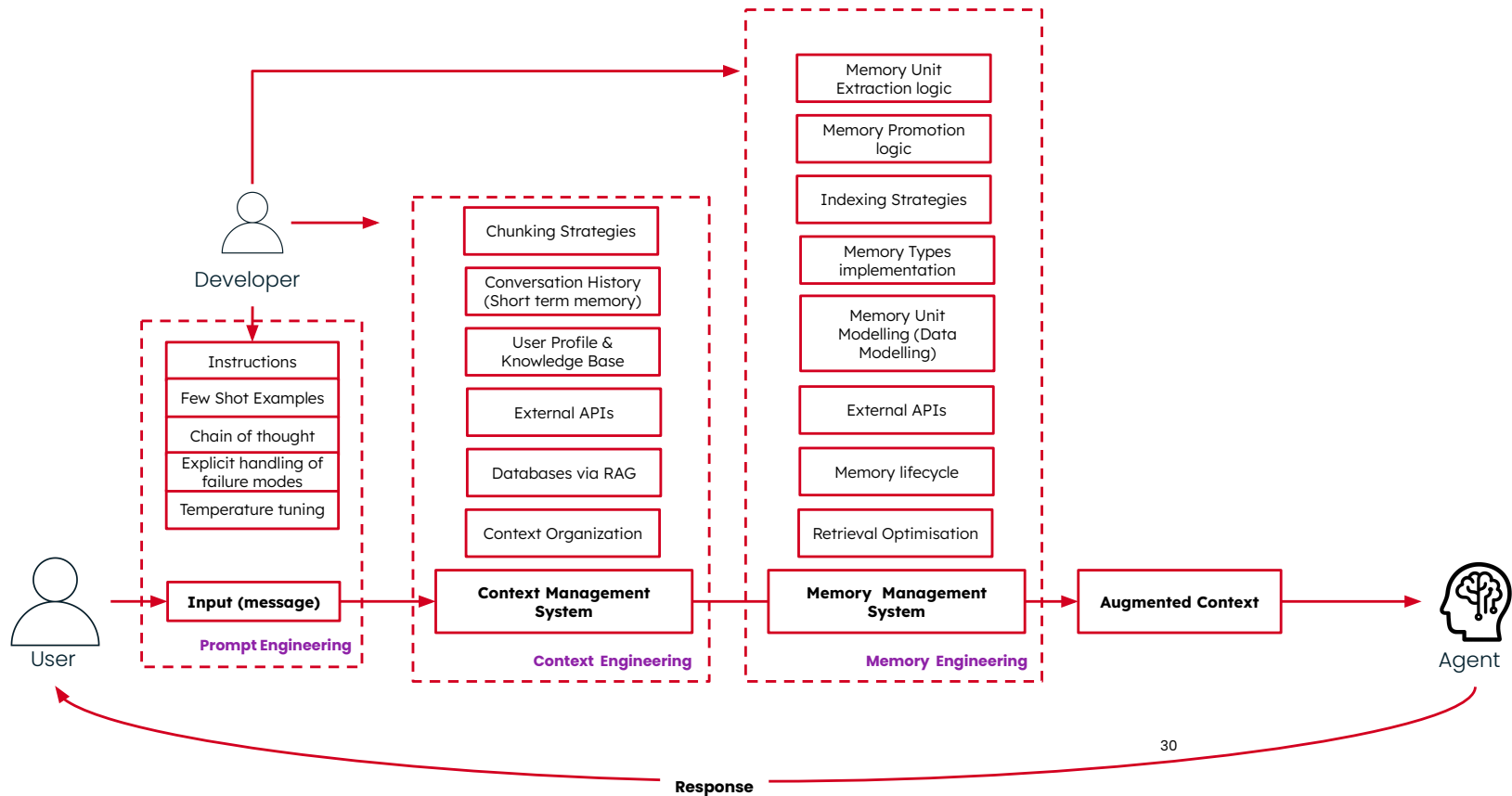
- Degradation of model performance as input length increases
- Naive retrieval mechanisms
- Passive, unstructured memory that prevents agent adaptation
- Building and maintenance of complex context pipelines
- Memory drift can degrade performance.

Memory Engineering: Beyond Context Engineering

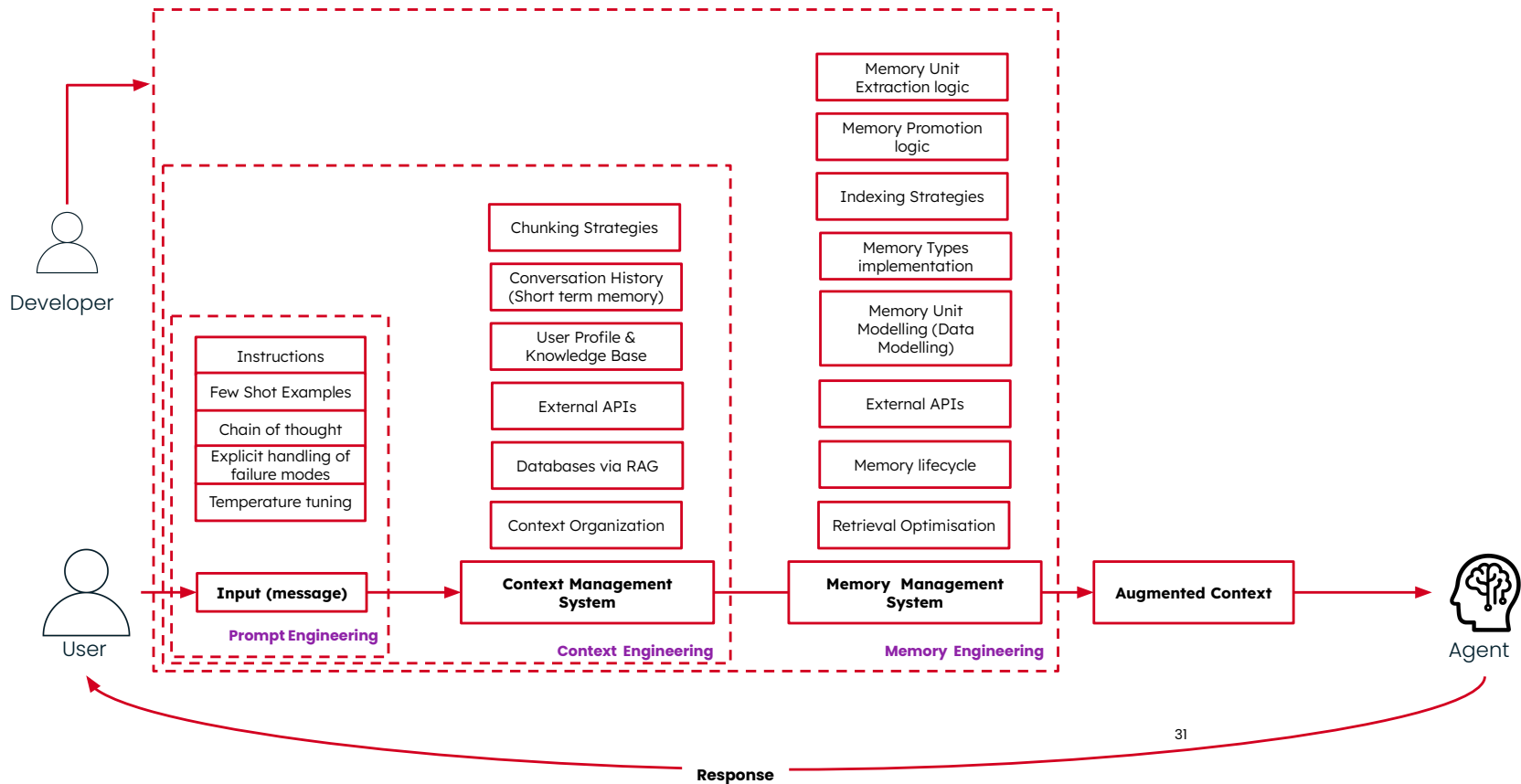
Overview of AI Memory

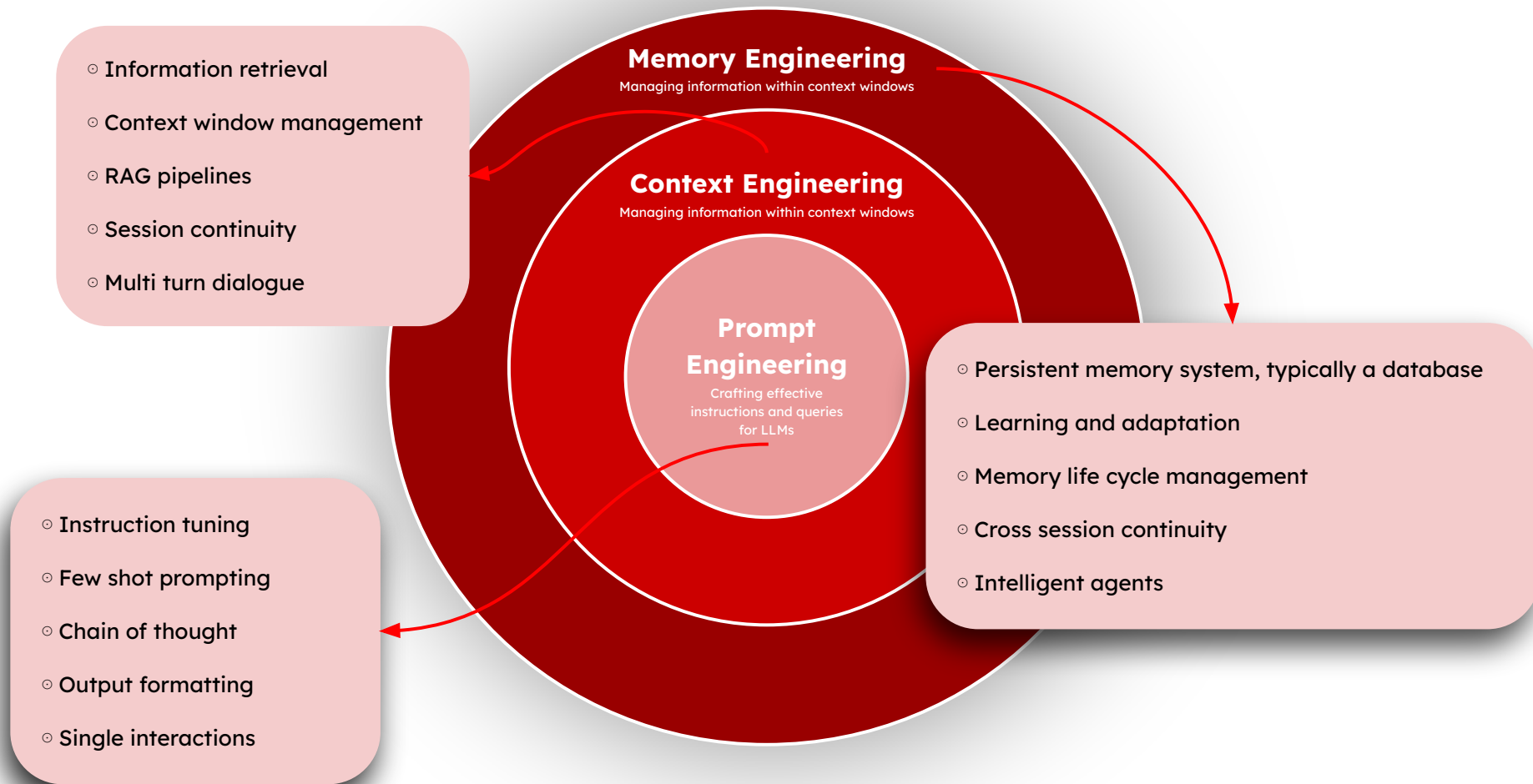
Agent Memory Engineering is the discipline focused on **designing and optimizing persistent memory** substrates, **retrieval mechanisms**, and **memory lifecycle management** strategies that enable AI agents to accumulate knowledge, maintain contextual awareness, and adapt behavior through sophisticated cognitive architectures.

Memory Engineering



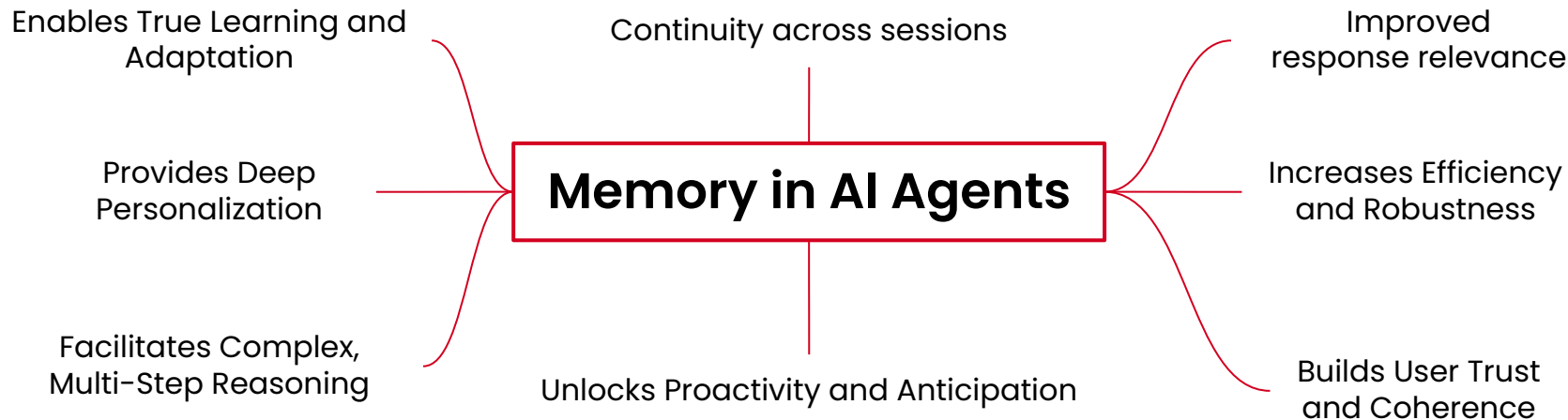
Memory Engineering





The Importance and Benefit of Memory in AI Agents

Overview of AI Memory



The Importance and Benefit of Memory in AI Agents

Overview of AI Memory



Introduction to Memory Management

Overview of AI Memory

Simple but understandable

Agent Memory Management is the dynamic and systematic process of making AI Agents more **Reliable**, **Believable** and **Capable**

How Memory Works in Human (Loosely)

Overview of AI Memory

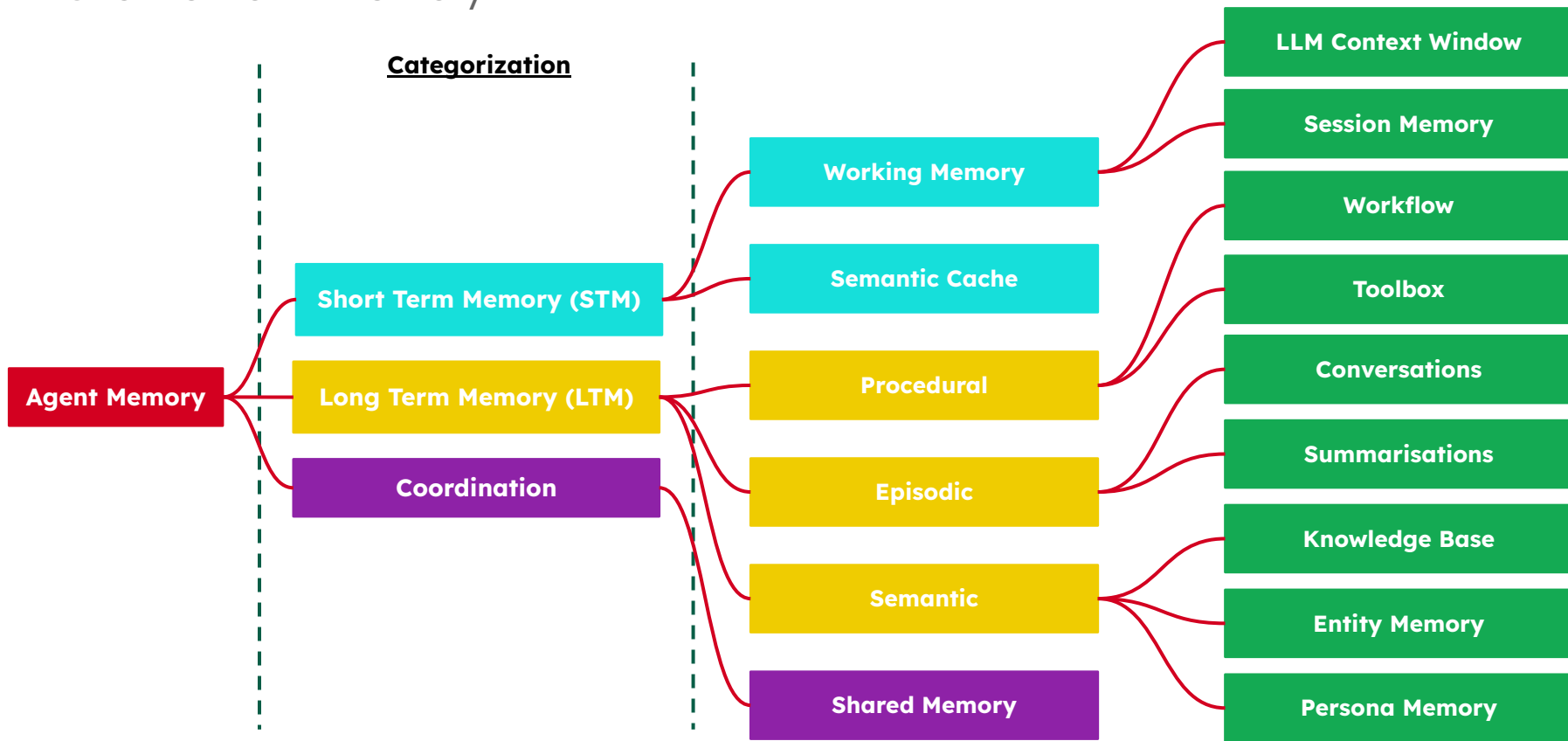
Humans have a good ability to **retain, recall** and **reuse** information over **short and long period** of time.





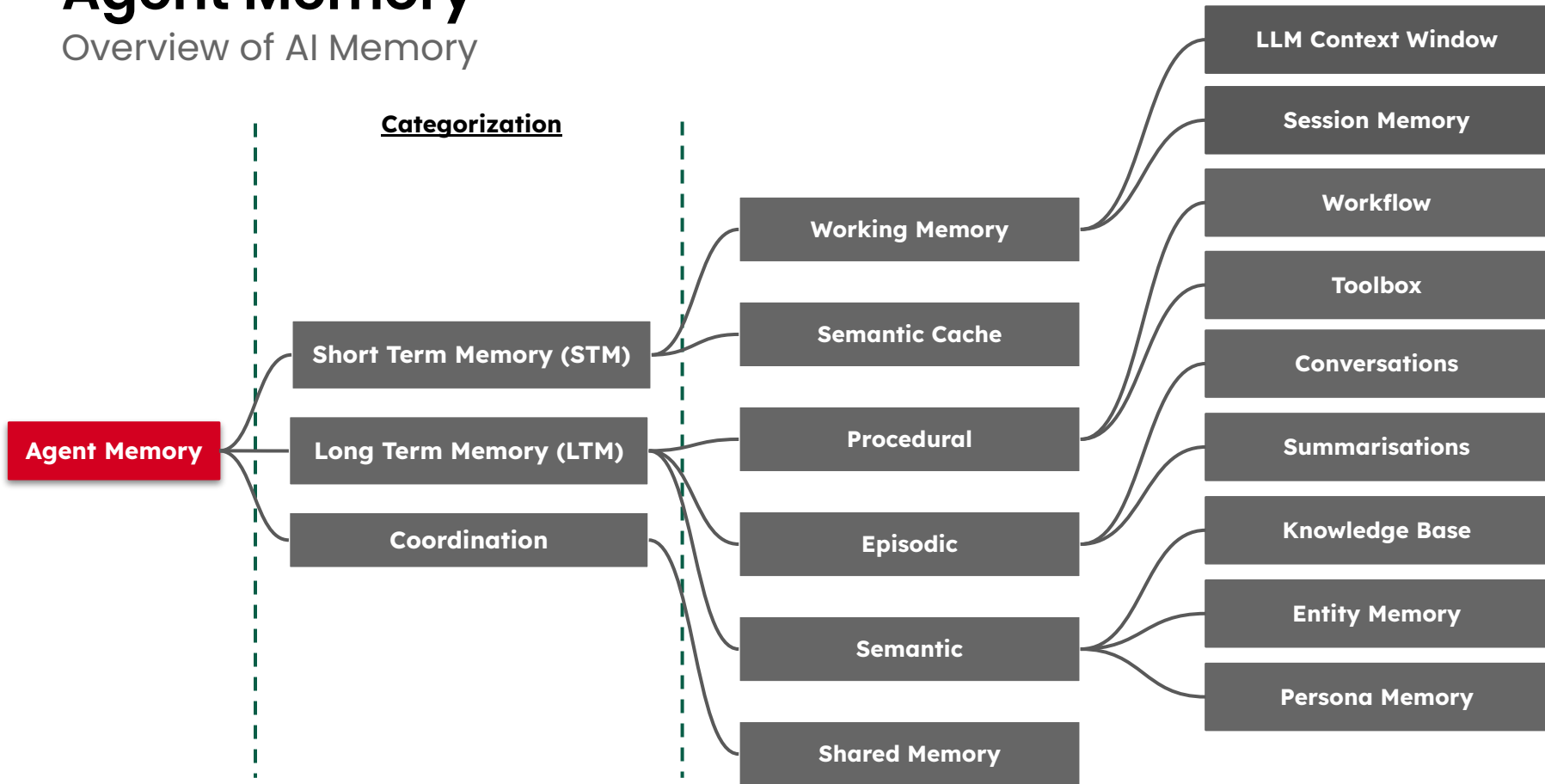
Forms of Memory in AI Agents (Memory Types)

Overview of AI Memory



Agent Memory

Overview of AI Memory



Agent Memory

Overview of AI Memory

Agent Memory is persistent memory management system that transforms stateless AI applications into intelligent agents capable of learning, adapting, and maintaining continuity across interactions.

Agent Memory combines the **LLM's inherent memory** (parametric knowledge in weights and context window) with **external persistent storage and memory management systems**, enabling agents to accumulate knowledge, maintain contextual awareness, and evolve behavior based on historical patterns.

LLM Memory

Internal to the model

Parametric Memory (Model Weights)

Knowledge encoded in the network parameters

- Pre-training knowledge
- Fine tuning adaptations
- Static during inference

Context Memory (Context Window)

Transient session information

- Current conversation
 - Token limited
 - Lost after session

Limitations

- Bounded context window
- Stateless Inference

Agent Memory

Agent Memory = LLM Memory + External Persistence +
Memory Management System

Persistent Memory Management

Knowledge accumulation & adaptation

- Short term & long term storage
 - Cross session continuity
 - Behavioural adaptation
 - Learning from experience
 - Multi modal retrieval
- Dynamic memory management

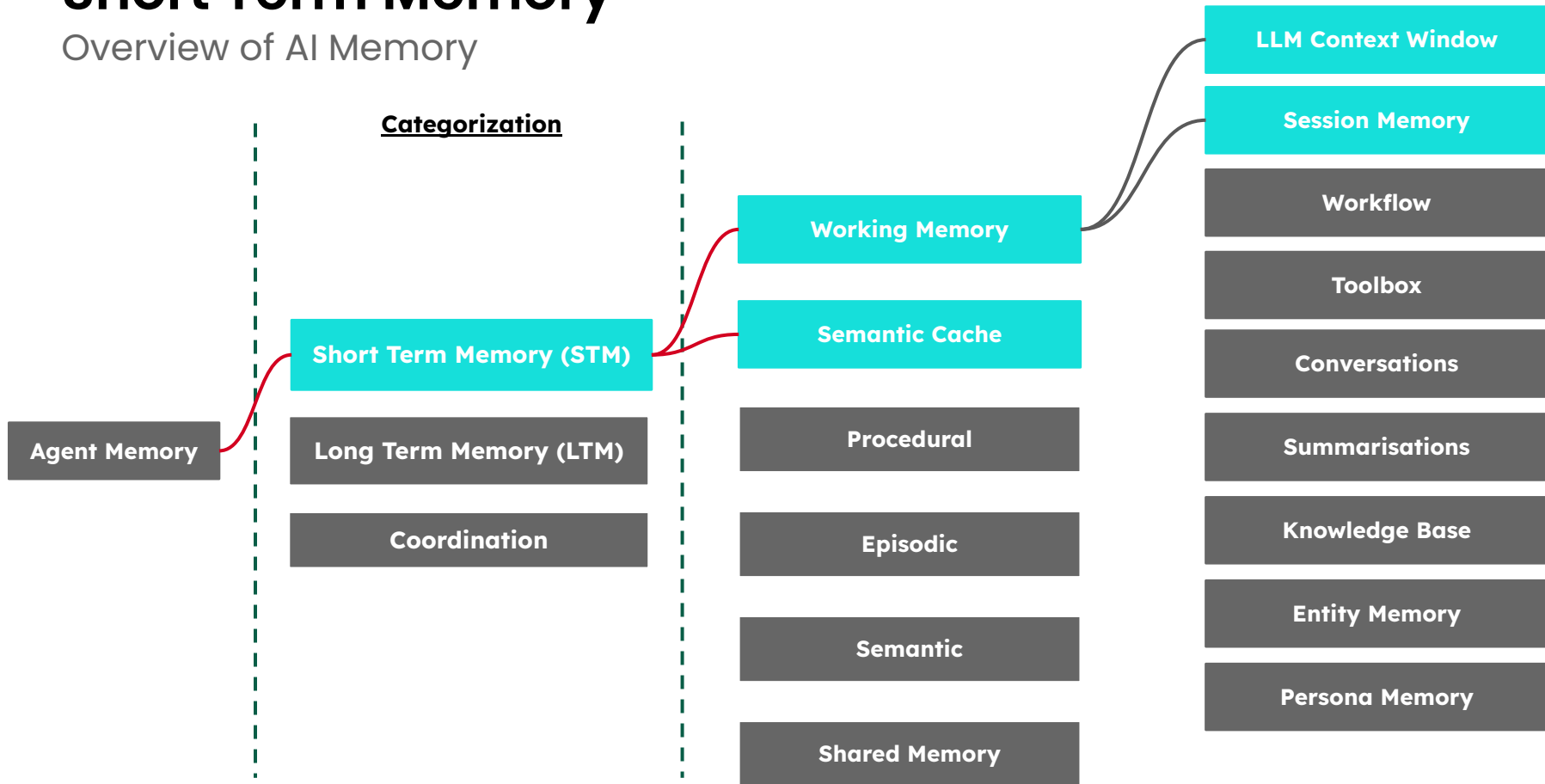
Enable True Intelligence

Reliable	Believable	Capable
Consistent context	Trustworthy interactions	Accumulated Knowledge

- **Memory Types:** Episodic, Semantic, Procedural Memory
 - **Examples:** Working Memory & Semantic Cache
 - Shared & Associative Memory

Short Term Memory

Overview of AI Memory



Short Term Memory

Overview of AI Memory

Short Term Memory is the information layer that agents use for **immediate task execution**, typically persisting from **seconds to hours**. STM can include working memory (the agent's active "scratchpad") and semantic cache (recent query-response pairs for rapid retrieval).

Working Memory

Overview of AI Memory

Working Memory is the agent's **temporary workspace** for manipulating information during active tasks, implemented as a partitioned context window area or session-based temporary storage. It acts as a "**scratchpad**" holding intermediate results, current observations, and in-progress computations

Context Window

Strategically managing what information remains in the active prompt during task execution.

LLM Context Window (8000 Tokens)

	System Prompt 500 tokens <u>Role:</u> Technical Support Agent <u>Instructions:</u> Be helpful, accurate, empathetic <u>Constraints:</u> Follow company policies

	Working Memory Segment 2500 tokens - Current Task: "Analyze Q3 earnings report for AAPL" - Company: APPL Sector: Technology

	Conversation History(Compressed) 2000 tokens - User: "What were apple's Q2 results?" - Agent: ""revenue was around... [Compressed]"

	Current Query 200 tokens What is the projected impact on the next quarter's earnings?

Available Space: ~ 2800 tokens



As Context Fills:

- Older working memory gets compressed into summaries
- Low priority information is flushed out of the context

Token Usage



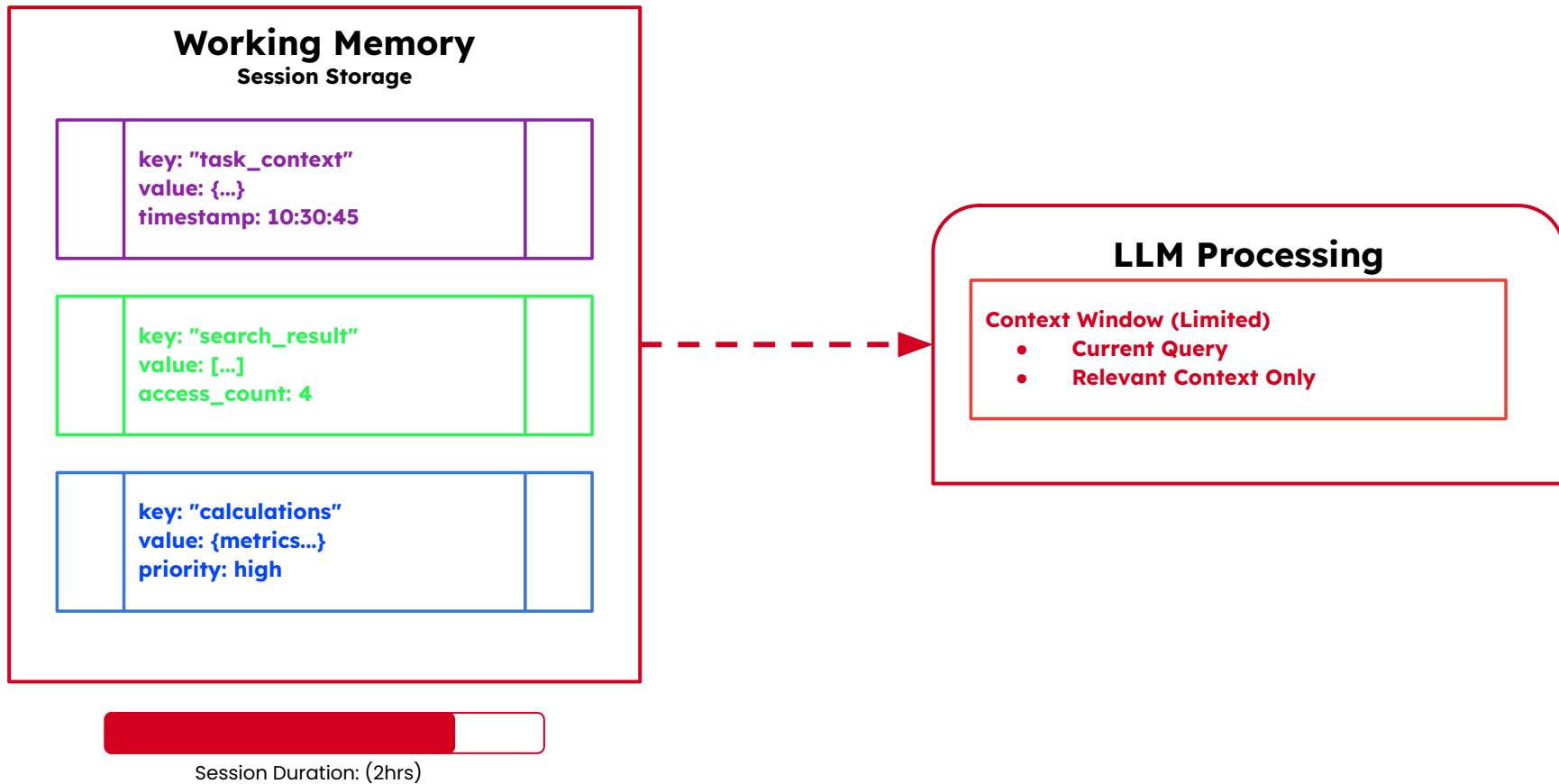
Working Memory

Overview of AI Memory

Working Memory is the agent's **temporary workspace** for manipulating information during active tasks, implemented as a partitioned context window area or session-based temporary storage. It acts as a "**scratchpad**" holding intermediate results, current observations, and in-progress computations

Temporary Session Storage

Persists throughout a user session but is cleared when the session ends



E-commerce Customer Support Agent Handling Complex Return

Customer Context: Sarah is contacting support about a complicated return involving multiple items from different orders placed over the past month. She's frustrated because she's already explained the situation twice to different agents.

Initial Contact (10:30:45)

Sarah says: *"Hi, I need help with returning several items. I spoke to someone yesterday about this but got disconnected."*

```
{
  key: "task_context",
  value: {
    customer_id: "CUST_89234",
    issue_type:
"multi_item_return",
    previous_contact: true,
    sentiment: "frustrated",
    priority_escalation: true
  },
  timestamp: "10:30:45"
}
```

E-commerce Customer Support Agent Handling Complex Return

Customer Context: Sarah is contacting support about a complicated return involving multiple items from different orders placed over the past month. She's frustrated because she's already explained the situation twice to different agents.

Information Gathering (10:31:20)

Agent: *"I apologize for the disconnection, Sarah. Let me pull up your complete order history and previous conversation."*

```
{
  key: "search_results",
  value: [
    {order_id: "ORD_7829", date: "2024-12-15", items: 5, status:
      "delivered"},
    {order_id: "ORD_7901", date: "2024-12-22", items: 3, status:
      "delivered"},
    {order_id: "ORD_8003", date: "2025-01-03", items: 2, status:
      "delivered"},
    {previous_ticket: "TKT_45231", agent: "James", unresolved: true}
  ],
  access_count: 3 // Accessed multiple times as agent
  cross-references
}
```


E-commerce Customer Support Agent Handling Complex Return

Customer Context: Sarah is contacting support about a complicated return involving multiple items from different orders placed over the past month. She's frustrated because she's already explained the situation twice to different agents.

Complex Calculation (10:32:00)

Sarah: *"I want to return 2 items from my December 15th order, 1 from December 22nd, and both items from January 3rd. Can you tell me the total refund?"*

```
{
  key: "calculations",
  value: {
    metrics: {
      total_items: 5,
      return_value: 347.82,
      shipping_credits: 15.00,
      loyalty_points_impact: -150,
      return_windows: {
        "ORD_7829": "expires_soon", // 3 days left
        "ORD_7901": "valid",        // 10 days left
        "ORD_8003": "valid"         // 24 days left
      }
    }
  },
  priority: "high" // High priority due to expiring return window
}
```

1. Sarah connects → Create session storage
2. Agent identifies complex issue → Store in task_context
3. System retrieves order history → Cache in search_results
4. Agent calculates refunds → Store in calculations
5. Throughout conversation → Selectively inject relevant memory into LLM
6. Resolution reached → Save important data to permanent storage
7. Session ends → Working memory cleared

Semantic Cache

Overview of AI Memory

Semantic Cache is a short-term memory system that stores recent query-response pairs indexed by semantic similarity rather than exact matches, enabling instant retrieval of cached responses for semantically equivalent queries.

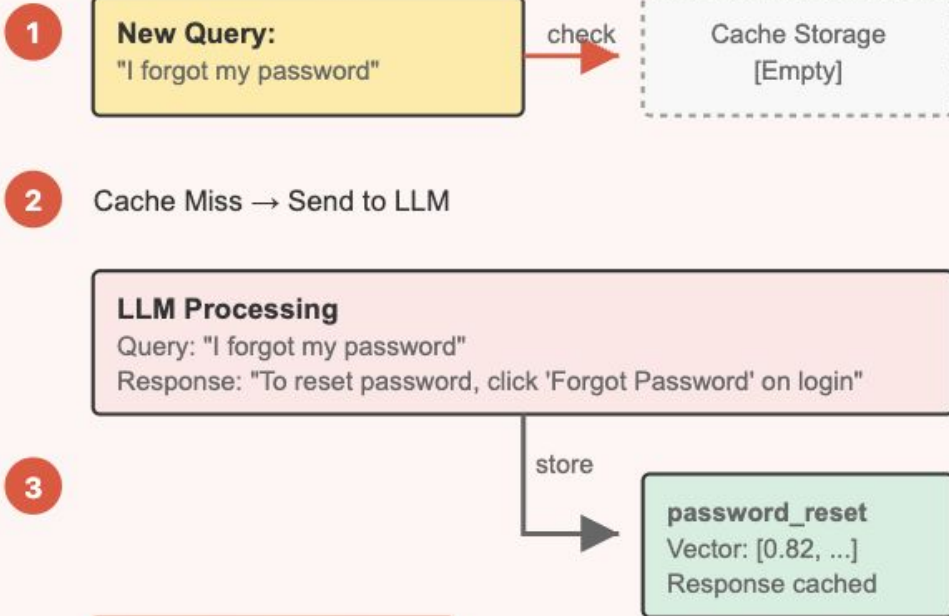
Semantic cache uses vector embeddings to identify when new queries have the same underlying meaning as previously processed ones

Allowing a customer support bot to instantly return the same answer for "forgot my password," "can't log in," and "need account reset" without reprocessing(LLM Inference)

Semantic Cache

Overview of AI Memory

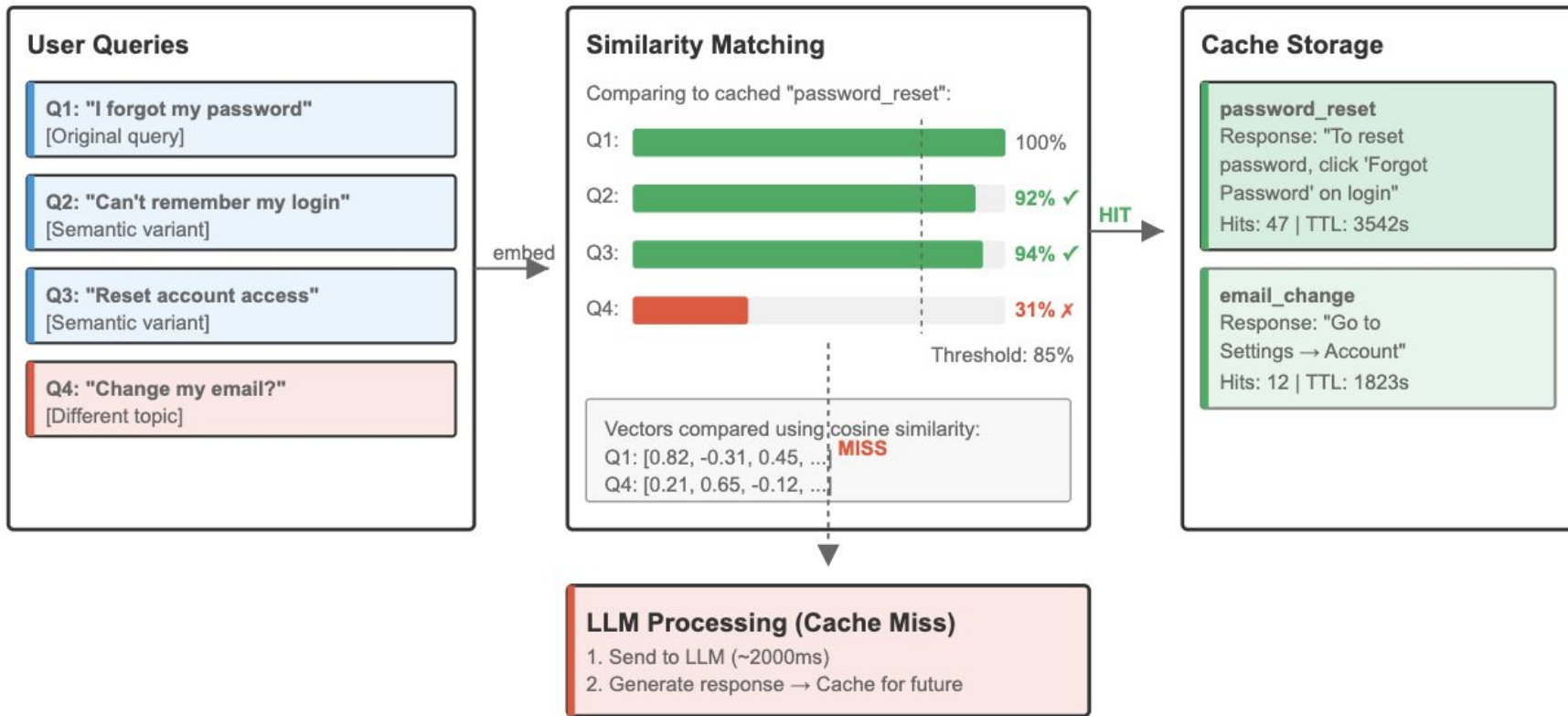
Phase 1: Initial Query (Cache Empty)



Total time: ~2000ms

Semantic Cache

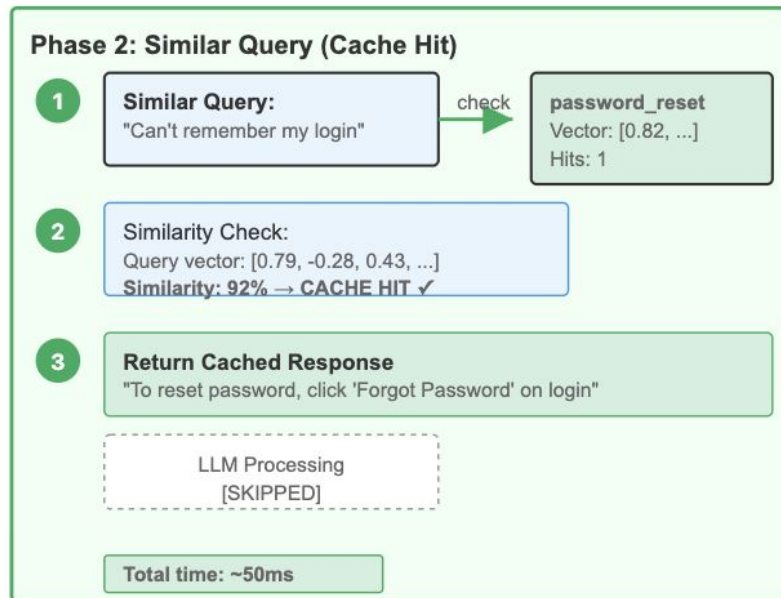
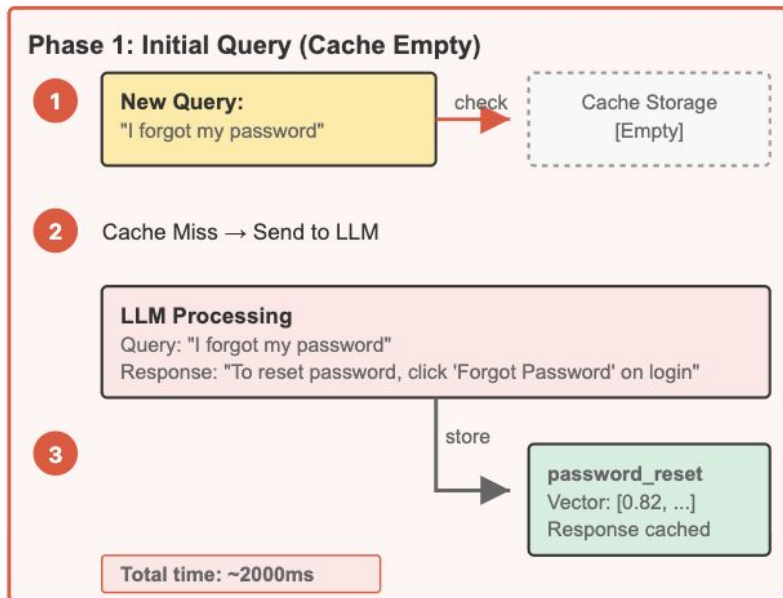
Overview of AI Memory



Semantic Cache

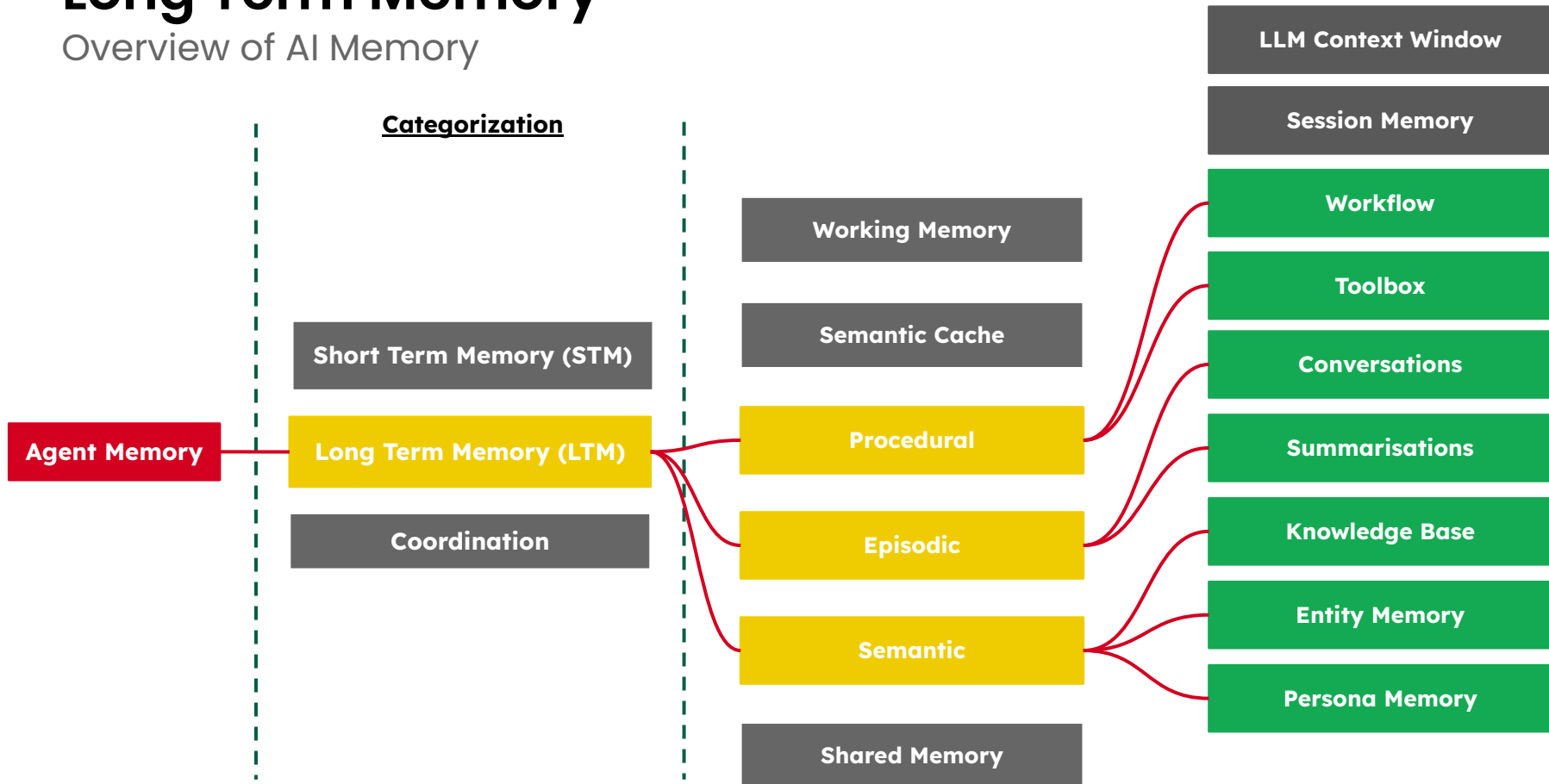
Overview of AI Memory

Semantic Cache Lifecycle



Long Term Memory

Overview of AI Memory



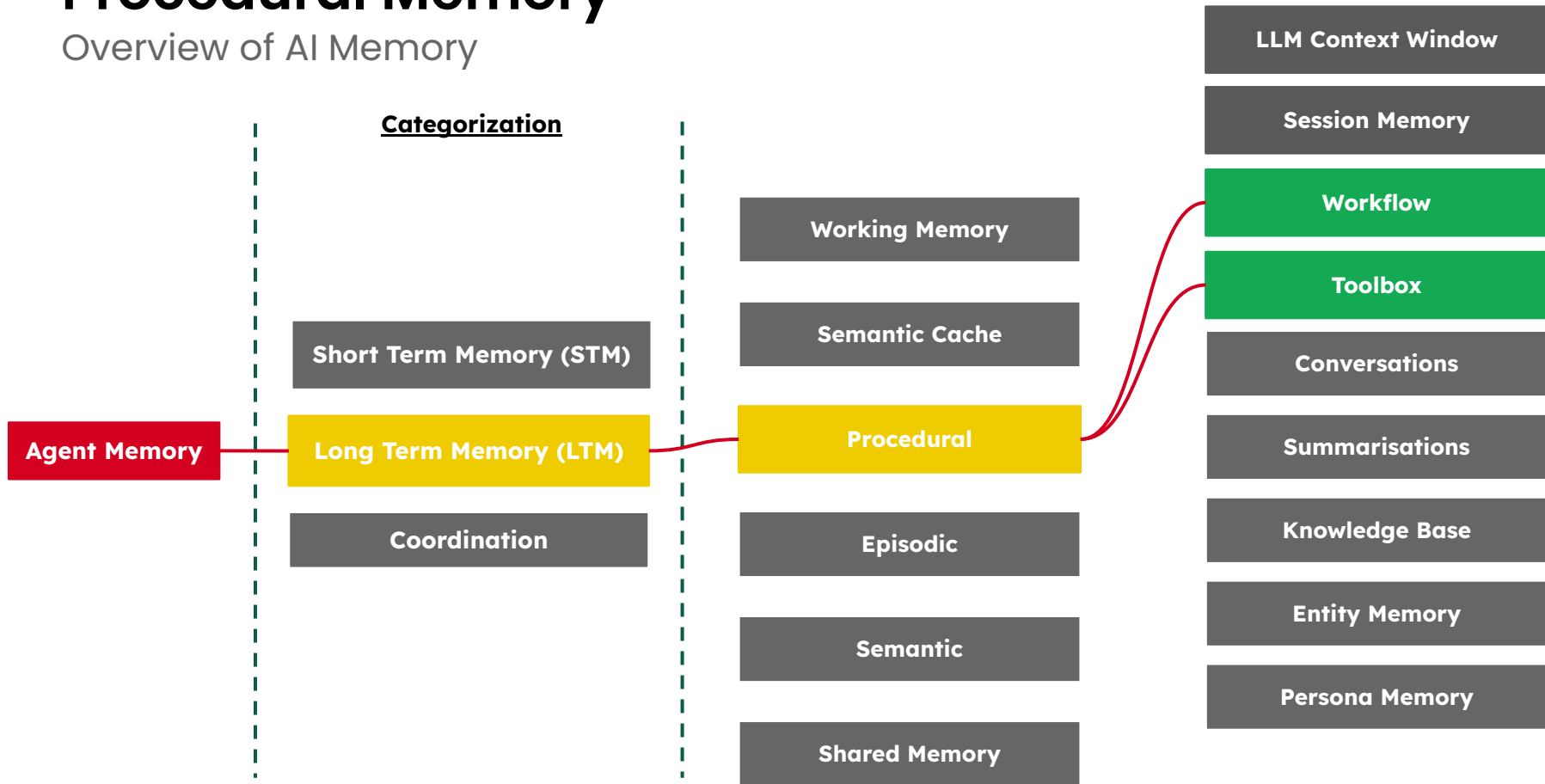
Long Term Memory

Overview of AI Memory

Long Term Memory is the persistent knowledge foundation that enables agents to maintain continuity, accumulate learning, and evolve capabilities **across sessions** and **extended time periods**.

Procedural Memory

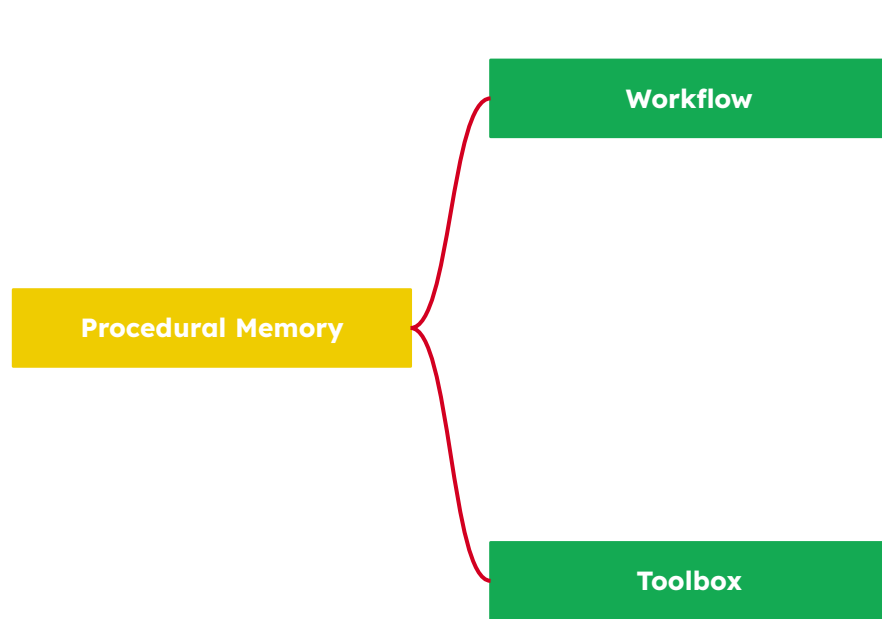
Overview of AI Memory



Procedural Memory

Overview of AI Memory

Procedural memory is the agent's repository of learned skills, methods, and multi-step processes that enable execution of complex tasks without explicit instructions. Procedural memory stores action sequences, decision trees, and execution patterns—allowing an agent to recall and apply established workflows.



Workflow Memory

Type of Procedural Memory in AI Agents

Workflow Memory is a specialized form of procedural memory that captures and stores complete execution traces of multi-step processes, including the sequence of actions taken, intermediate results, decision points, and outcomes.

Each workflow memory unit contains tool invocations, parameters used, timestamps, and error states, enabling agents to learn from past executions, optimize future runs, and recover from failures by referencing previously successful paths.

Workflow Memory

Type of Procedural Memory in AI Agents

Real-Time Workflow Execution

1 search_database
args: {query: "Q3 sales"}
✓ Success: 142 records

2 calculate_metrics
args: {data: records}
✓ Success: metrics computed

3 generate_report
args: {metrics: result}
⌚ Processing...

Message History

```
messages.append({  
  "type": "function_call_output",  
  "call_id": call.call_id,  
  "output": str(result),  
})
```

extract

Workflow Status

Name: "Q3 Sales Analysis Workflow"
Steps Completed: 2/3 | Current: generate_report
Outcome: IN_PROGRESS

Memory Unit Extraction

Step 1 Memory Unit

_id: "tool_8f3a2b1c"
arguments: {query: "Q3"}
result: "142 records"
timestamp: "10:32:15"

Step 2 Memory Unit

_id: "tool_9c4d3e2f"
arguments: {data: ...}
result: "metrics"
timestamp: "10:32:47"

Step 3 Memory Unit

_id: "tool_7a5b6c3d"
arguments: {metrics: ...}
result: [pending]
timestamp: "10:33:12"

Complete Workflow Memory

Embedding: [0.34, -0.67, 0.89, ...] for semantic retrieval
Storage: MemoryType.WORKFLOW_MEMORY

Workflow Memory

Type of Procedural Memory in AI Agents

Future Reference & Learning

New Query:

"Generate Q4 sales report"

retrieve

Retrieved Similar Workflow

Previous: "Q3 Sales Analysis Workflow" (95% similarity)

Learned Pattern:

1. search_database → 2. calculate_metrics → 3. generate_report

Optimizations: Use index on date field, cache calculations

✓ Reuse successful workflow pattern

Benefits:

- Skip trial & error
- Apply learned optimizations
- Avoid past failures
- Consistent execution

Time saved: ~40%

Workflow Memory

Type of Procedural Memory in AI Agents

Workflow Memory is a specialized form of procedural memory that captures and stores complete execution traces of multi-step processes, including the sequence of actions taken, intermediate results, decision points, and outcomes.

Workflow Memory Process Summary:

1. Execute tools capturing each step
2. Extract metadata into memory units
3. Store complete workflow with embedding
4. Retrieve & apply patterns for similar tasks



Toolbox Memory

Type of Procedural Memory in AI Agents

Toolbox Memory is the agent's registry of available tools, functions, and capabilities with their associated metadata, usage patterns, and performance characteristics.

This memory stores not just what tools exist, but how to use them effectively—including parameter configurations, success rates, typical execution times, and learned optimizations—allowing agents to dynamically discover and select the right tool for a given task through semantic search rather than hard-coded mappings.

Toolbox Memory

Type of Procedural Memory in AI Agents

Tool Registration Process

```
@toolbox.register_tool
def search_database(query):
    """Search customer DB"""
    return db.find(query)
```

extract

Tool Metadata

```
name: search_database
docstring: "Search
customer DB"
signature: (query)
_id: ObjectId()
augmented: false
```

Embedding Generation

```
embedding = get_embedding(f"{name} {docstring} {signature}")
Result: [0.34, -0.67, 0.89, 0.23, -0.45, 0.76, ...]
```

Persistent Storage

MemoryType.TOOLBOX

In-Memory Functions

_tools[id] = callable

Semantic Tool Discovery

Agent Task:

"Find all customers who ordered in Q3"

vector search

Toolbox Search Results

search_database

95% match ✓

filter_by_date

88% match ✓

calculate_revenue

40% low

Threshold: 70% - Returns top relevant tools only

Toolbox Memory

Type of Procedural Memory in AI Agents

Dynamic Tool Execution

Selected Tools:

1. search_database
2. filter_by_date

**Auto-discovered based
on task semantics**

No manual selection!

Execution Pipeline:

```
tool_1 = _tools["5f8a3b2c"] # Retrieve callable
```

```
results = tool_1(query="Q3 orders") # Execute
```

```
filtered = tool_2(results, start="2024-07-01")
```

Benefits:

- Scales to 1000s of tools
- No hardcoded tool mapping
- Discovers relevant tools
- Adapts to new tools

100% dynamic discovery

Toolbox Memory

Type of Procedural Memory in AI Agents

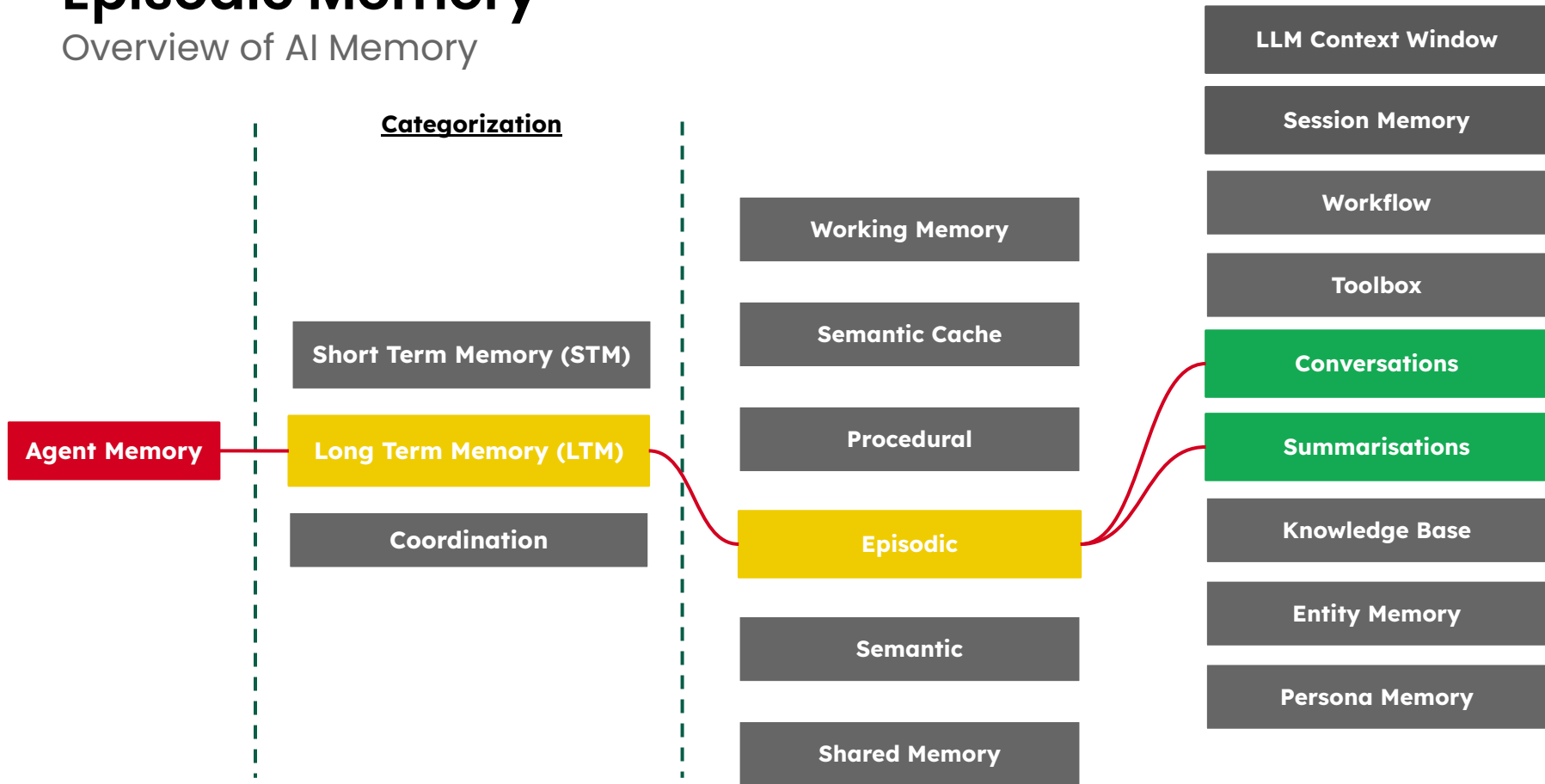
Toolbox Memory is the agent's registry of available tools, functions, and capabilities with their associated metadata, usage patterns, and performance characteristics.

Toolbox Memory Process

1. **Register:** Developer adds `@toolbox.register_tool`` decorator to any Python function.
2. **Embed:** System converts function name, description, and parameters into a vector `[0.34, -0.67, ...]`.
3. **Store:** Save tool metadata/embedding in database and keep callable function in memory.
4. **Task Arrives:** Agent receives task "Find Q3 customers" and converts it to a vector.
5. **Search:** Compare task vector to all tool vectors, finding matches above 70% similarity.
6. **Execute:** Retrieve and run discovered tools like `search_database()` and `filter_by_date()`.
7. **Complete:** Task accomplished without hardcoding—tools discovered purely by semantic understanding.

Episodic Memory

Overview of AI Memory

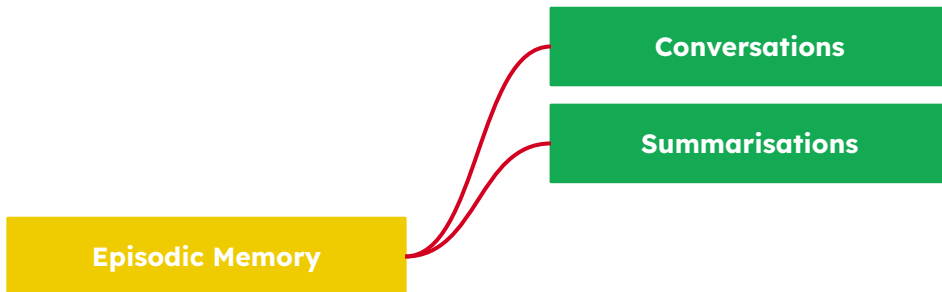


Episodic Memory

Overview of AI Memory

Episodic Memory is the agent's chronological record of specific events, interactions, and experiences, analogous to human autobiographical memory.

Each memory unit captures the full context of an interaction including participants, timestamps, outcomes, and emotional valence, enabling agents to reference specific past events ("remember when you helped me debug that issue last Tuesday?") and maintain relationship continuity across sessions.





Conversational Memory

Type of Episodic Memory in AI Agents

Conversational Memory is a subset of episodic memory specifically focused on dialogue history, storing complete conversation transcripts with speaker turns, topic transitions, and contextual metadata.

This enables agents to maintain conversational coherence, reference earlier discussions, track commitments made during conversations, and adapt communication style based on historical interaction patterns with specific users.

Conversational Memory

Type of Episodic Memory in AI Agents

Active Conversation (Tuesday, Oct 15, 10:30 AM)

U

Sarah (User):

"I'm having trouble with the login API again,"
"same issue as last Tuesday"

Agent:

"I remember helping you with the authentication timeout."
"Let me check if it's the same 401 error pattern"

A

U

Sarah:

"Yes! That's exactly it. You had me check the"
"token expiration settings"

Conversation Metadata:

session_id: "conv_8f3a2b1c"
participants: ["sarah_wilson", "support_agent"]
timestamp: "2024-10-15T10:30:00Z"
topic: "API authentication issue"

Memory Unit Extraction

Episode Memory Unit (Current)

episode_id: "ep_2024_10_15_001"
participants: {user: "sarah_wilson", agent: "support_01"}
timestamp: "2024-10-15T10:30:00Z"
speaker_turns: 3
outcome: "issue_identified"
emotion: "frustrated → relieved"

Retrieved Referenced Memory

"remember...same issue as last Tuesday"
episode_id: "ep_2024_10_08_003"
date: "2024-10-08 (last Tuesday)"
issue: "401 authentication timeout"
solution: "Extended token TTL from 1h to 24h"
outcome: "resolved_successfully"
sentiment: "satisfied"

store



Summarisation Memory

Type of Episodic Memory in AI Agents

Summarization Memory is the compressed representations of longer interactions or documents that preserve key insights while reducing storage and retrieval overhead

Rather than storing complete transcripts, summarization memory maintains distilled versions that capture essential facts, decisions, and outcomes—allowing agents to quickly reference the gist of past interactions without processing full historical records.

Summarisation Memory

Type of Episodic Memory in AI Agents

Original Full Transcript (2-hour meeting)

10:00 AM - John: "Let's discuss the Q3 product roadmap..."
10:02 AM - Sarah: "I think we should prioritize the API redesign..."
10:05 AM - John: "Good point, but what about customer feedback..."
10:08 AM - Mike: "The analytics show 67% want feature X..."
10:12 AM - Sarah: "We could do both if we adjust timeline..."
10:15 AM - John: "Let me share the budget constraints..."
10:18 AM - Lisa: "Engineering says 3 months for API..."
10:22 AM - Mike: "Customer success reports 42 tickets about..."
10:25 AM - Sarah: "What if we phase the rollout..."
10:28 AM - John: "That could work, let's look at dependencies..."

...

[1,847 more lines]

...

11:58 AM - John: "So we're agreed on the phased approach?"
11:59 AM - All: "Yes, let's proceed with that plan."

Storage Metrics:

• Size: 142 KB | Tokens: ~35,000 | Retrieval time: ~

compress
LLM + RAG

Compressed Summary

Key Facts:

- Meeting: Q3 Product Roadmap Planning
- Duration: 2 hours | Participants: 4
- Main topics: API redesign, Feature X, Budget
- Customer data: 67% want Feature X, 42 tickets

Decisions Made:

1. Implement phased rollout approach
2. Prioritize API redesign (3 months)
3. Feature X in Phase 2

Action Items:

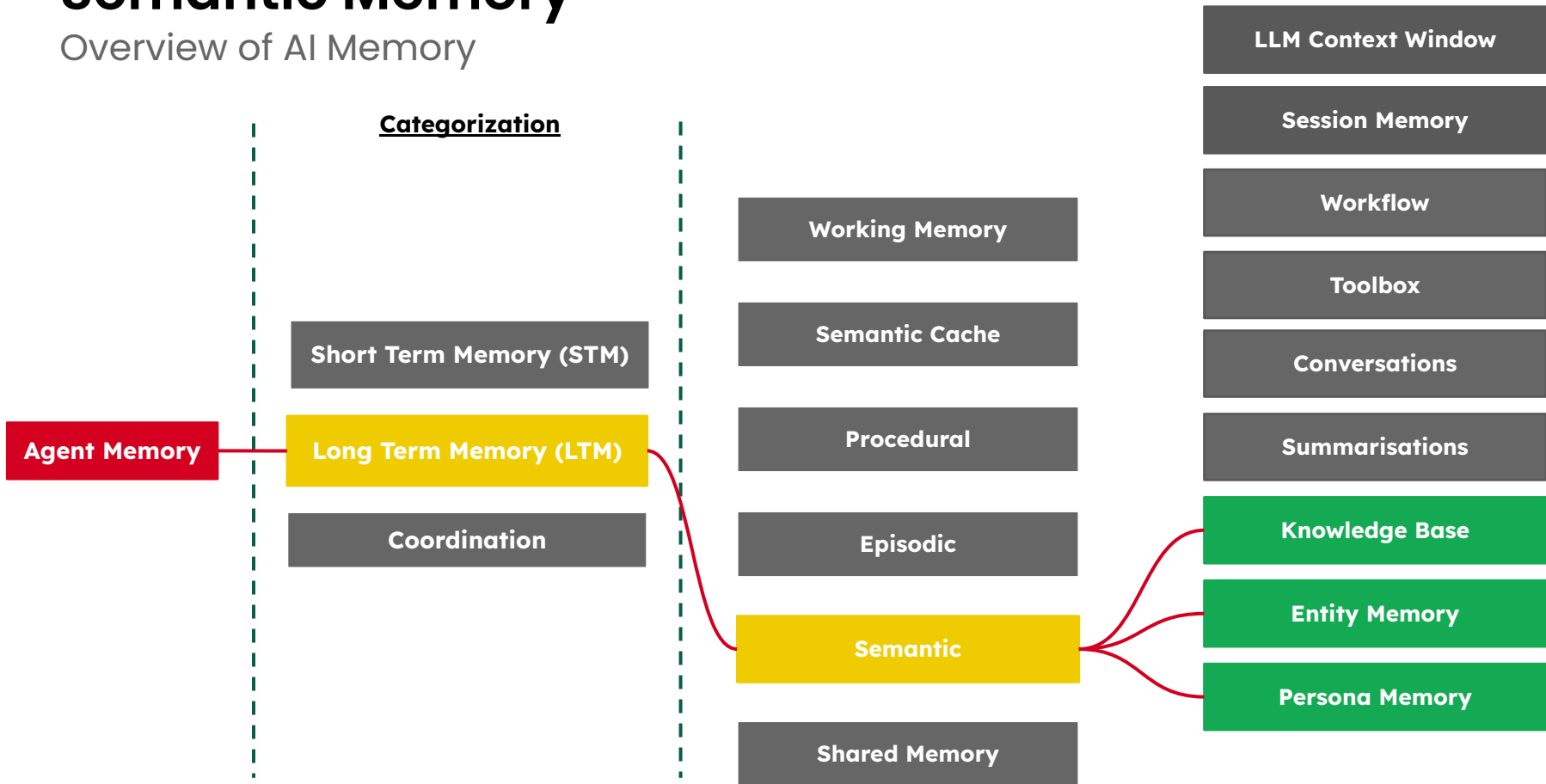
- Sarah: Draft phased timeline by Friday
- Mike: Prioritize customer tickets for API

Compressed Metrics:

• Size: 2.8 KB | Tokens: ~500 | Retrieval time: ~

Semantic Memory

Overview of AI Memory

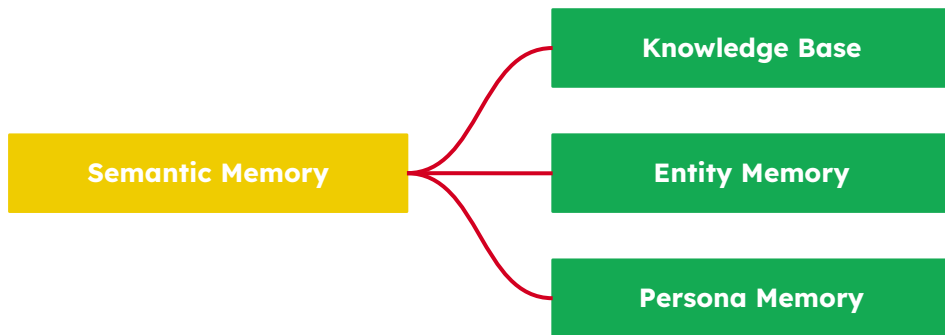


Semantic Memory

Overview of AI Memory

Semantic Memory is the agent's structured repository of facts, concepts, and general knowledge independent of specific experiences or temporal context.

This includes domain knowledge, definitions, relationships between concepts, and learned facts about the world—serving as the agent's "encyclopedia" that provides consistent, factual grounding for reasoning and decision-making across all interactions.





Knowledge Base

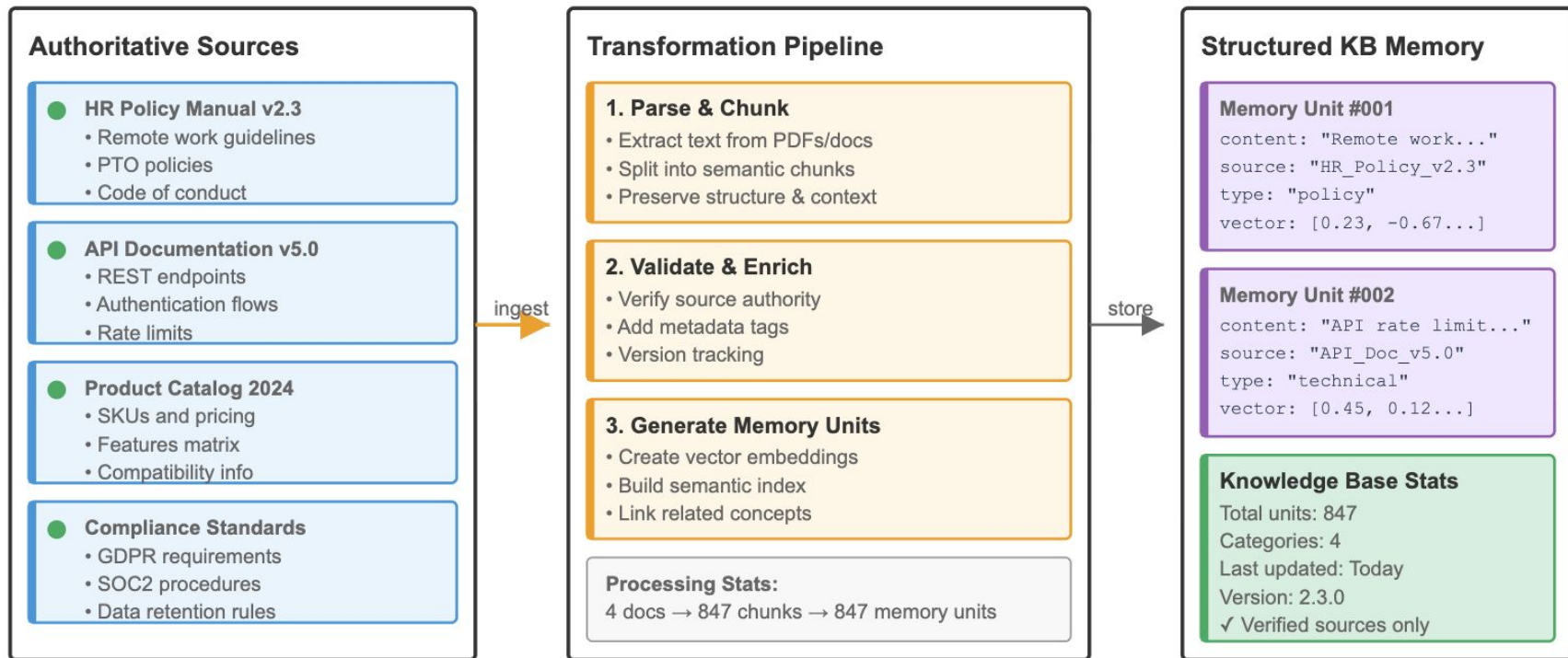
Type of Semantic Memory in AI Agents

Knowledge Base is a formally organized component of semantic memory containing verified, structured information typically ingested from authoritative sources.

Knowledge bases are often pre-populated with domain expertise, documentation, policies, and reference materials that provide the foundational truth the agent operates from, such as company policies, technical specifications, or scientific facts.

Knowledge Base

Type of Semantic Memory in AI Agents



Knowledge Base

Type of Semantic Memory in AI Agents

Agent Retrieval Process

User Query:

"Can I work from abroad
for 2 weeks?"

search

Semantic Search in KB:

Query → Vector → Similarity
Found: 3 relevant policies

Retrieved Knowledge:

From: HR_Policy_v2.3 § 4.2

"Remote work from international locations"

"requires manager approval for periods"

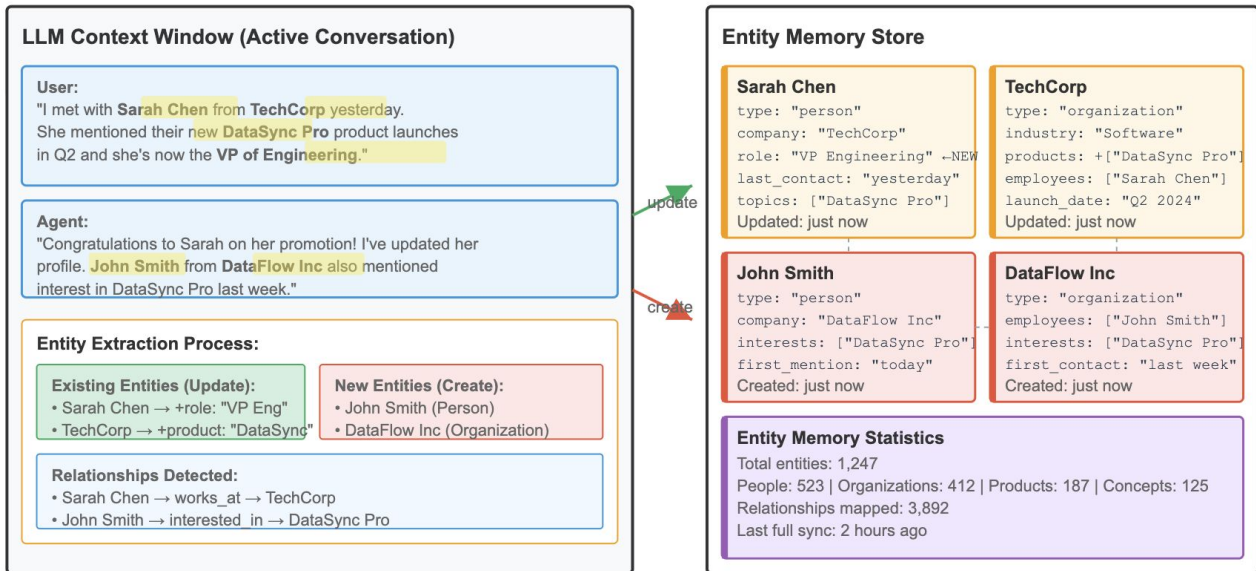
"up to 30 days. Tax implications apply."

✓ Authoritative answer

Entity Memory

Type of Semantic Memory in AI Agents

Entity Memory maintains detailed profiles of specific entities (people, organizations, products, concepts) including their attributes, relationships, and historical interactions. This memory enables personalized interactions and relationship-aware responses in AI agents interactions..



Entity Memory

Type of Semantic Memory in AI Agents

Personalized Response Using Entity Memory

Future Query:

"Set up a meeting with Sarah"

retrieve

Retrieved Entity Profile:

Sarah Chen

- VP Engineering at TechCorp
- Last discussed: DataSync Pro launch
- Timeline: Q2 2024
- Related: John Smith interested

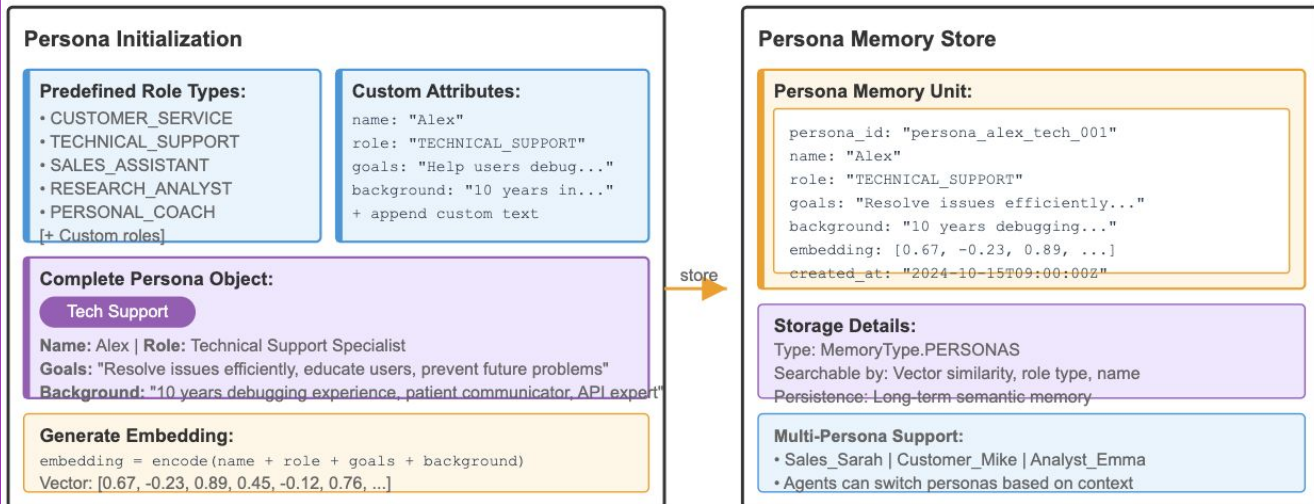
Context-Aware Response:

"I'll schedule a meeting with Sarah Chen (VP Engineering at TechCorp). Should we include DataSync Pro Q2 launch on the agenda?"

Perona Memory

Type of Semantic Memory in AI Agents

Persona Memory stores the encoded behavioral patterns, communication style, and role-specific knowledge that defines its consistent personality and expertise.



Perona Memory

Type of Semantic Memory in AI Agents

Active Persona in Agent

MemAgent.persona

Active: "Alex (Tech Support)"
Loaded at session start
Conditions all responses

Behavioral Influence:

- ✓ Communication style: Patient
- ✓ Knowledge focus: Technical
- ✓ Response format: Step-by-step
- ✓ Vocabulary: Technical terms
- ✓ Goal alignment: Problem-solving

Persona Evolution

update_persona_from_summaries()

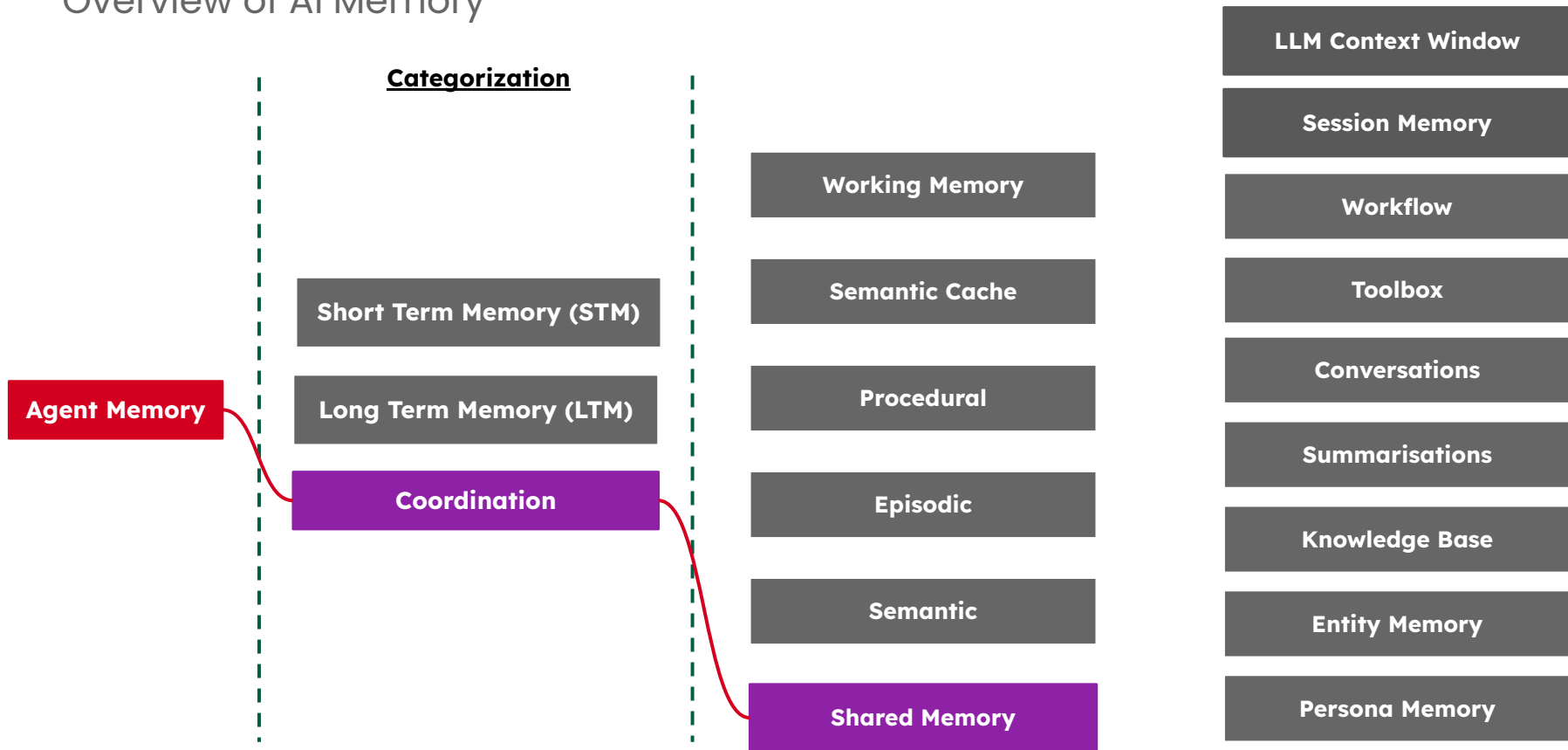
Triggered by interactions

LLM Updates:

Goals: +["Focus on API v2"]
Background: +["Handled 500+ auth issues"]
→ Regenerate embedding
→ Store updated version

Forms of Memory in AI Agents (Memory Types)

Overview of AI Memory



Introduction to Memory Management

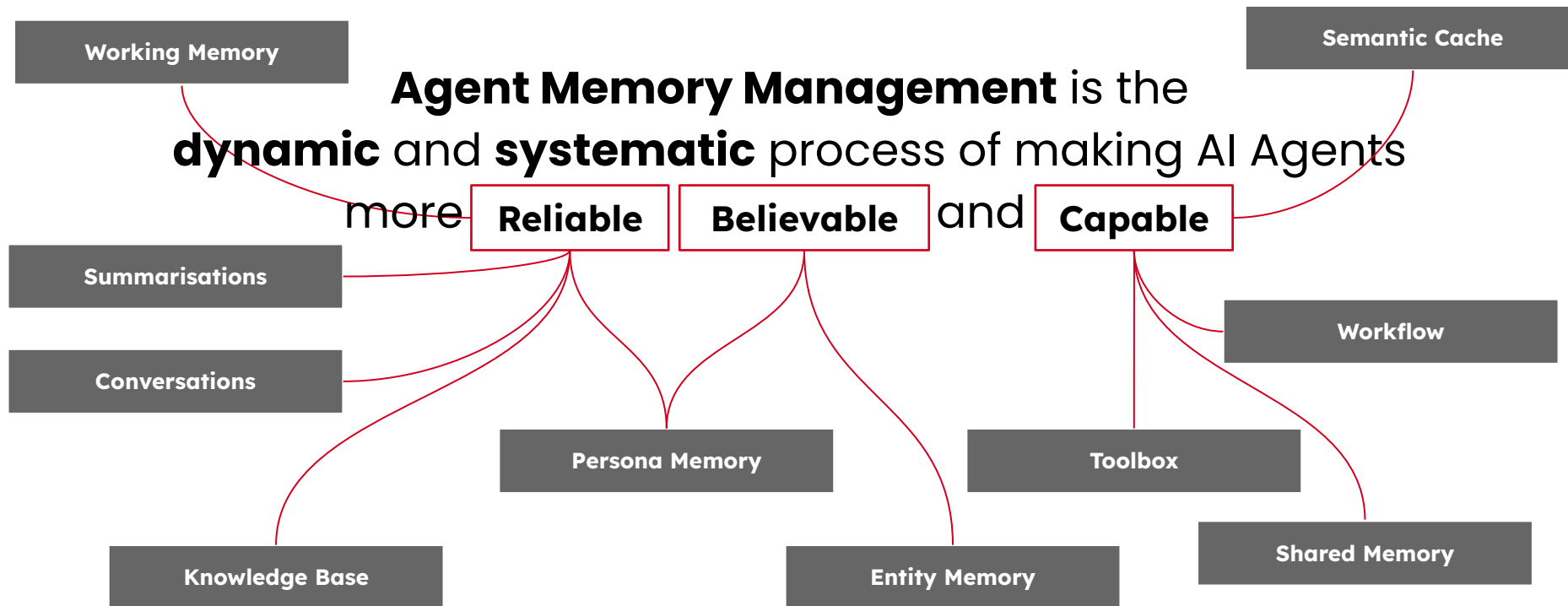
Overview of AI Memory

Simple but understandable

Agent Memory Management is the dynamic and systematic process of making AI Agents more **Reliable**, **Believable** and **Capable**

Memory Management

Overview of AI Memory



Memory Management

Overview of AI Memory

Purpose and Objective

Agent Memory Management is a computational **exocortex** for AI agents—a **dynamic, systematic** process that integrates an agent's **LLM memory** (context window and parametric weights) with a persistent **memory management system** to encode, store, retrieve, and synthesize experiences. Inspired by human cognitive memory, it enables agents to **accumulate knowledge**, maintain conversational and task **continuity**, and **adapt** behavior based on history—making them more **reliable, believable, and capable**.

The How

Outcome

Memory Management Is Systematic

Overview of AI Memory

1 Generation

2 Storage

3 Retrieval

4 Integration

5 Updating

6 Deletion

Memory management is systematic because **memory IS the system** that transforms stateless LLMs into intelligent agents

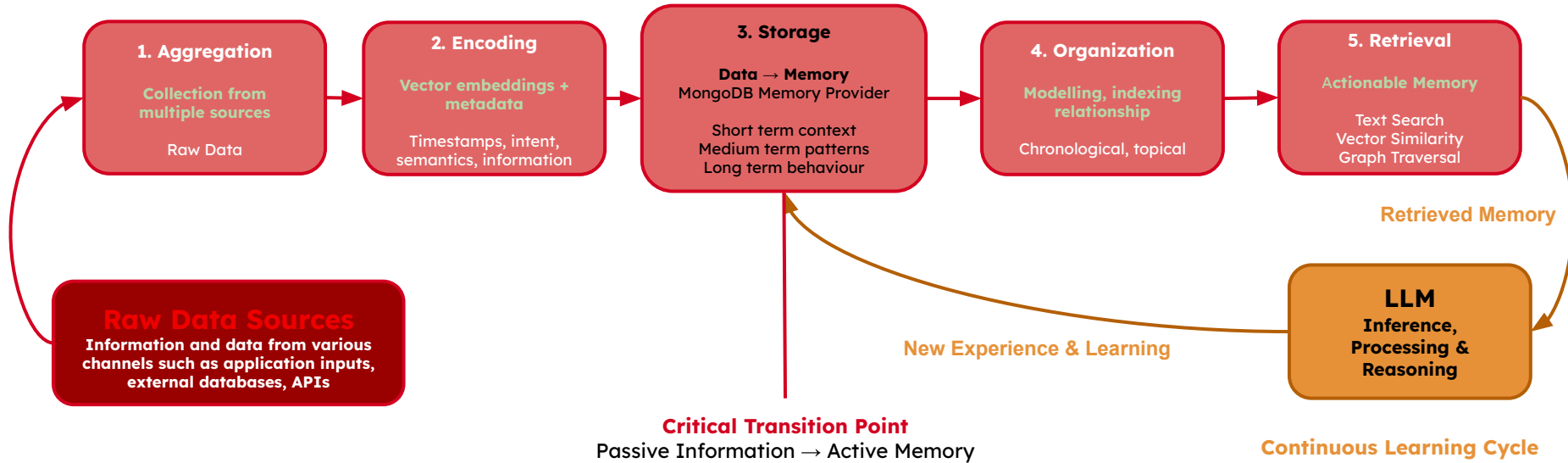
Without systematic coordination:

Retrieval might surface outdated information that should have been consolidated

Storage might duplicate information already in semantic memory

Forgetting might remove critical context needed for current operations

Data to Memory Transformation Pipeline



Memory Characteristics

Persistent

Survives Sessions and Interactions

Contextually Aware

Understands relevance and relationship

Adaptive

Influence future behaviour and reasoning

Retrievable

Available when needed for decisions

Forms of Memory in AI Agents (Memory Types)

Overview of AI Memory

1 Generation

2 Storage

3 **Retrieval**

4 Integration

5 Updating

6 ~~Deletion~~ **Forgetting**

Information **Retrieval**
(and everything in between)
Is All That Matters

Information Retrieval Mechanisms are systematic methods for **extracting and processing data from diverse sources**—such as PDFs, documents, databases, data warehouses, REST APIs, and the internet—for specific downstream operations.

These mechanisms encompass the **transformation of raw data into structured formats** through processes like document representation and indexing, followed by efficient query processing and ranking.

Poll Question

Question: Which of the following best describes Agent Memory?

- A) A way for AI agents to store only conversation history for one session
- B) A persistent memory system that lets agents retain, recall, and reuse knowledge across interactions
- C) A database designed only for storing LLM prompts
- D) A tool for increasing an agent's computational speed



Poll Question

Question: How familiar were you with AI memory concepts before this session?

- A) Very familiar – I've implemented them before
- B) Somewhat familiar – I know the basics
- C) I've heard of them but not in detail
- D) Completely new to me





Break

Module 2: Memory Management

- Information Retrieval & RAG Overview
- Lexical, Vector & Hybrid Search
- Introduction to Agentic RAG
- Embedding Models & Re-Rankers
- Core Concepts in Memory Engineering
- Retrieval Methods, Memory Types & Engineering
- Memory Architectures:
Generative Agents,
A-Mem, MemGPT
Memory Bank
- Design Patterns & Best Practices for Agentic Systems



Information Retrieval

1 Generation

2 Storage

3 **Retrieval**

4 Integration

5 Updating

6 ~~Deletion~~ **Forgetting**

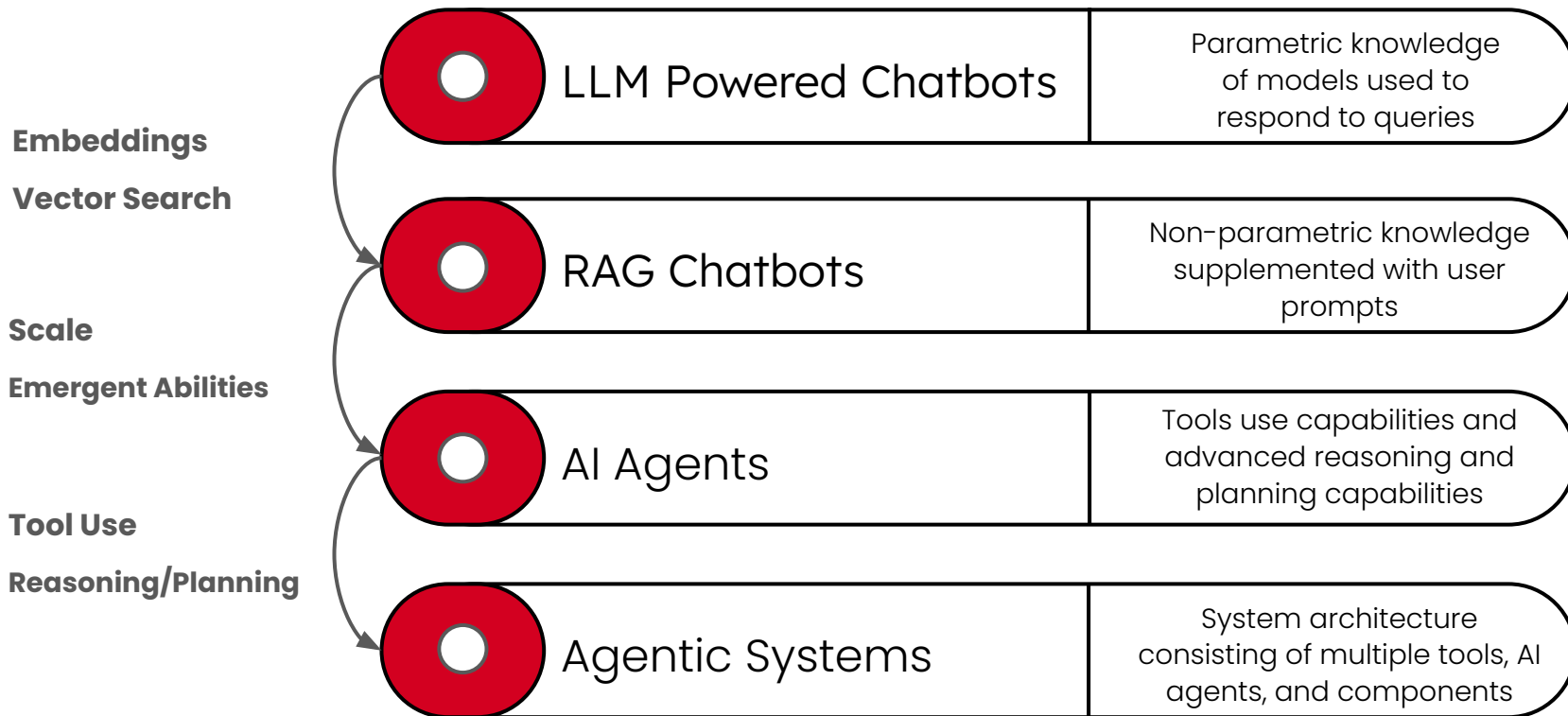
Information Retrieval
(and everything in between)
Is All That Matters

Information Retrieval Mechanisms are systematic methods for **extracting and processing data from diverse sources**—such as PDFs, documents, databases, data warehouses, REST APIs, and the internet—for specific downstream operations.

These mechanisms encompass the **transformation of raw data into structured formats** through processes like document representation and indexing, followed by efficient query processing and ranking.

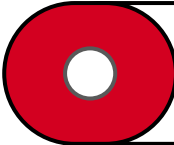
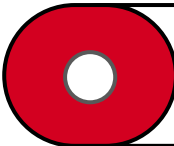
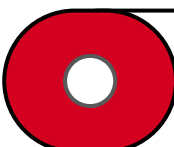
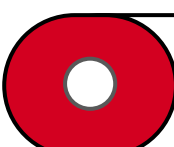
Form Factor Evolutions of GenAI Applications

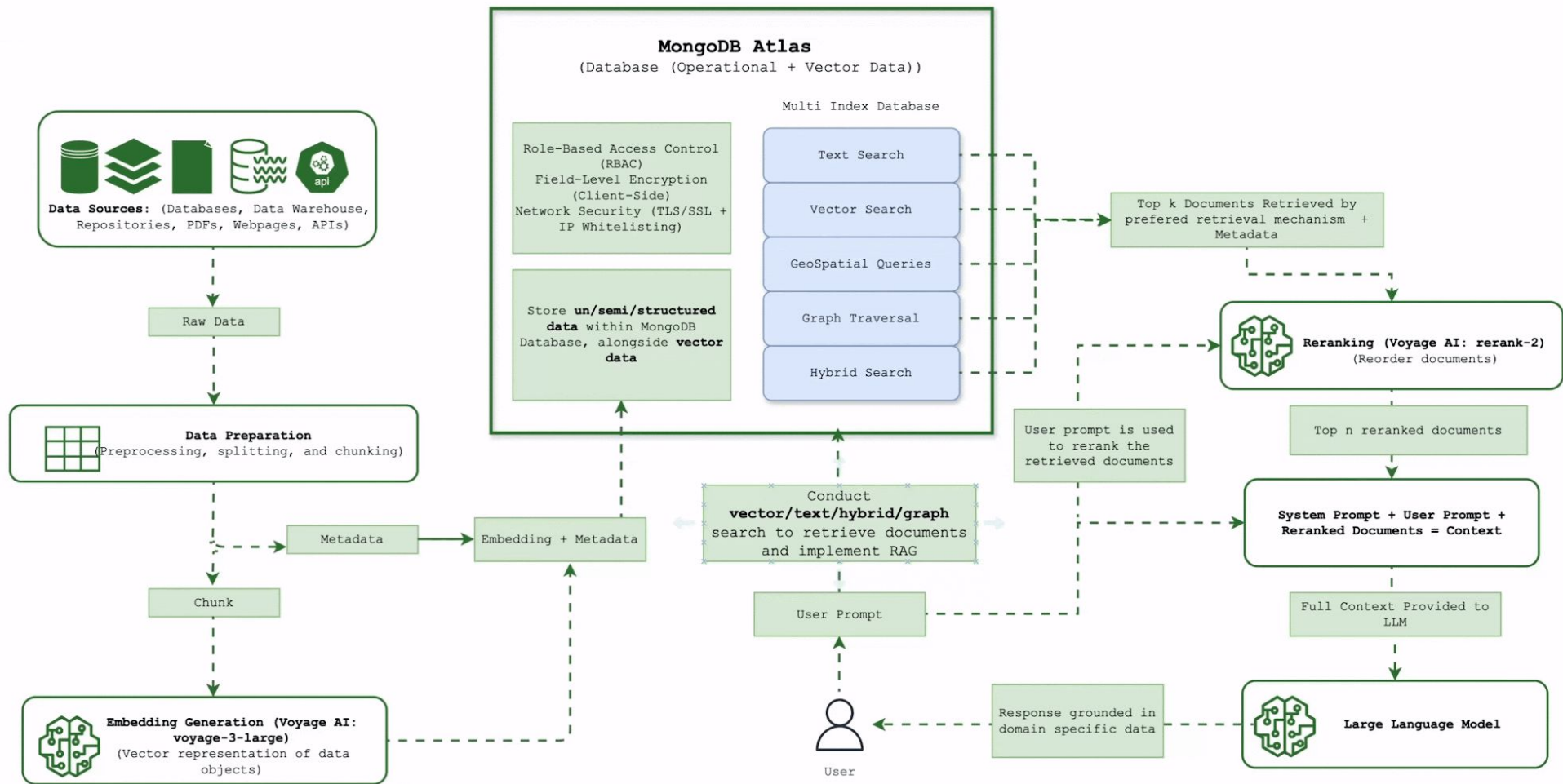
Memory Management



Common Methods of Information Retrieval

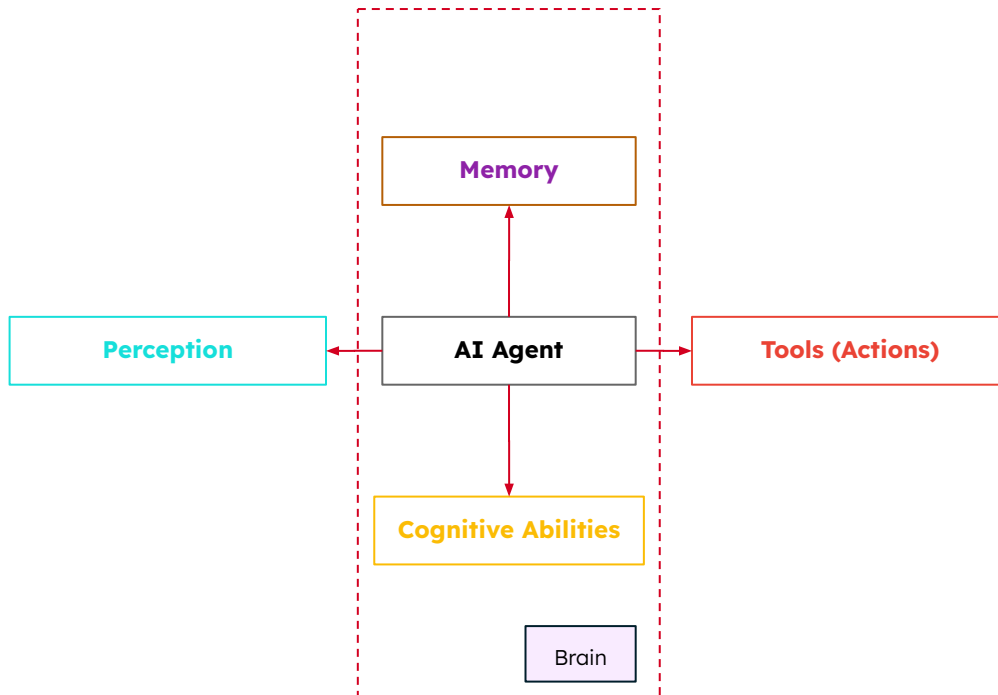
Information Retrieval

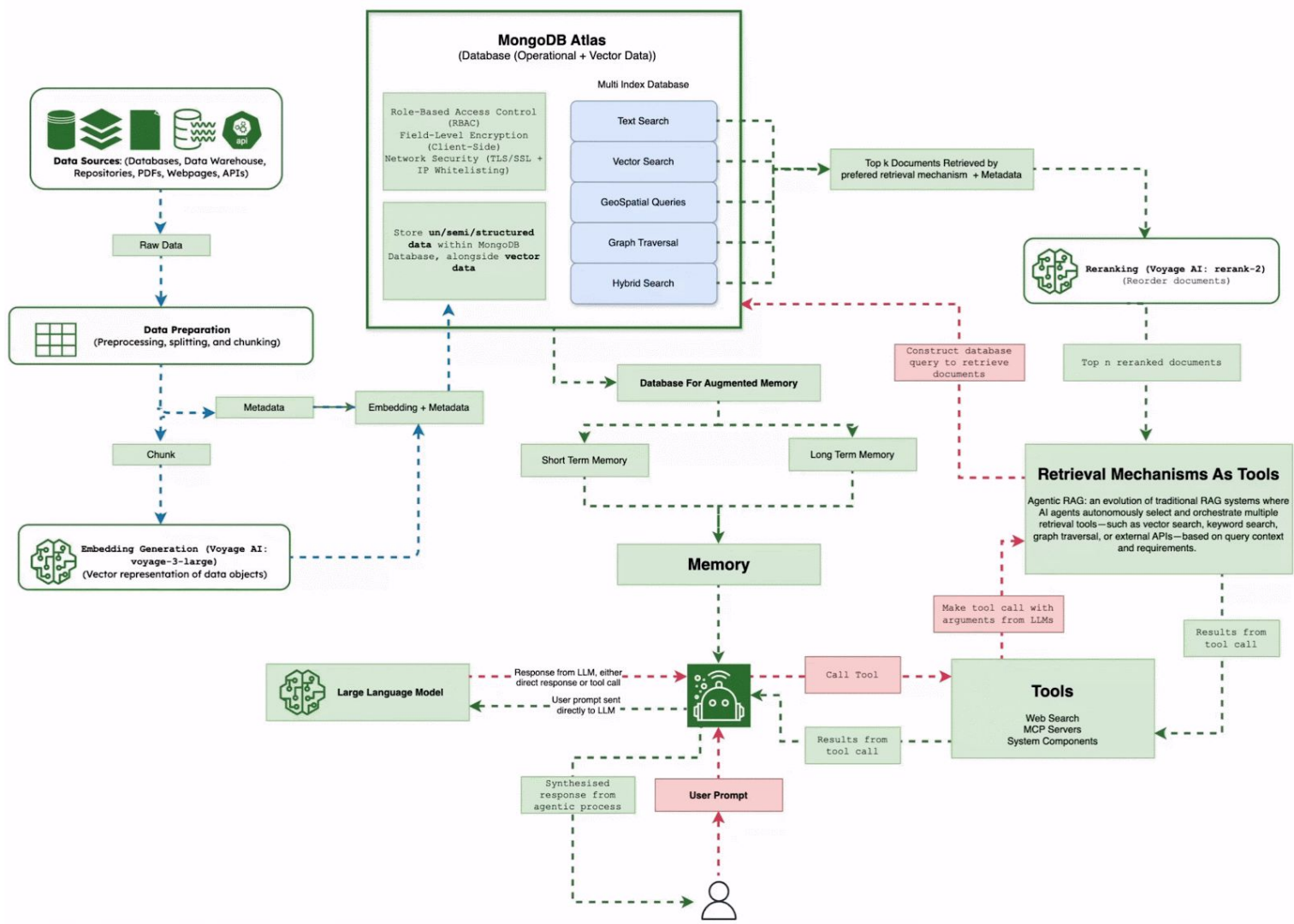
 Lexical Search	keyword-based retrieval using inverted indexes, term frequency, and exact or fuzzy string matching to find documents containing the query terms.
 Vector Search	Retrieves documents by comparing high-dimensional embeddings: queries and corpus items are mapped into the same vector space, and nearest neighbors (semantically closest) are returned.
 Hybrid Search	Combines vector and lexical approaches—first filters by keyword matching, then ranks results by embedding similarity (or vice versa) to balance precision and semantic recall.
 Agentic RAG	An architecture where one or more autonomous “agents” orchestrate retrieval, reasoning, tool invocation, and iterative refinement—dynamically fetching and integrating information into generated responses.



What Is An AI Agent?

Overview of AI Memory



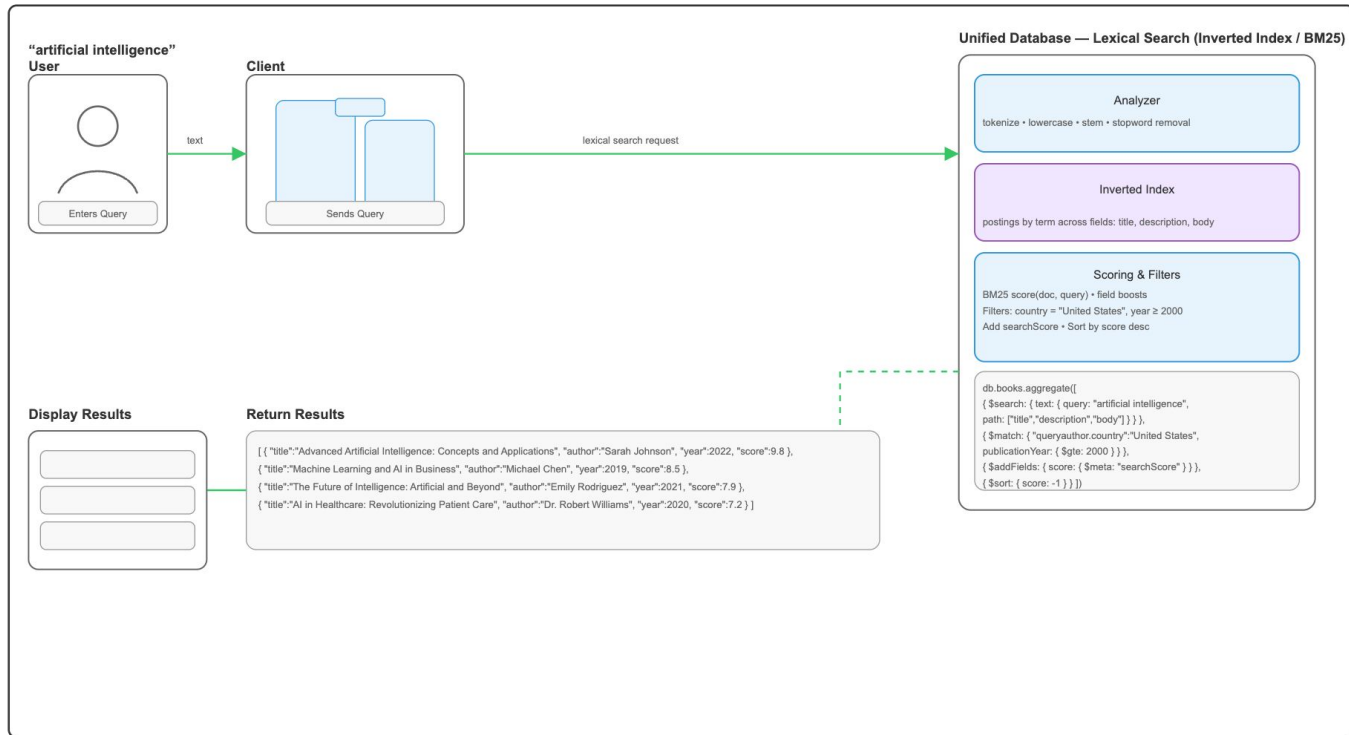


Lexical Search

Information Retrieval

Lexical Search Flow: Enter Query → Client Sends → Inverted Index (BM25) → Results → Display

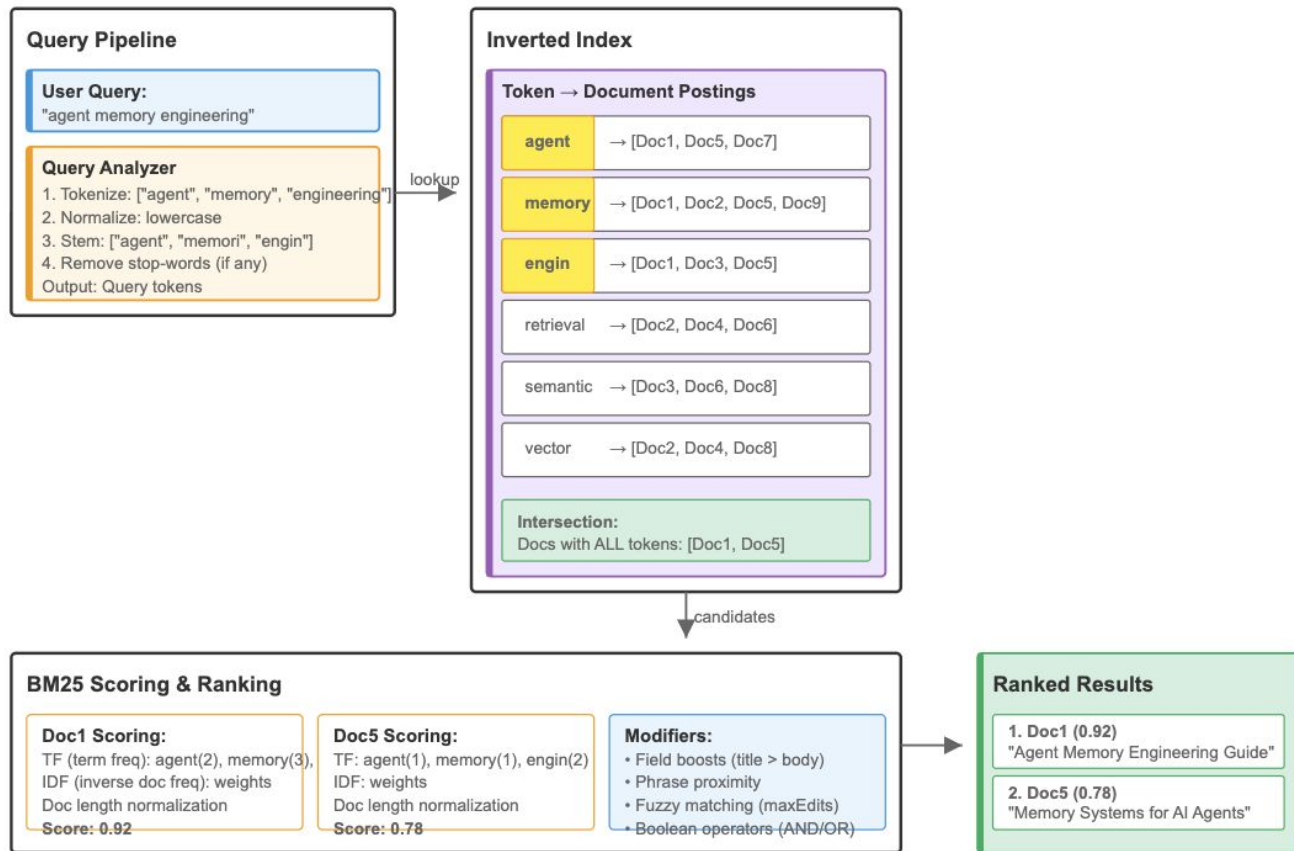
Lexical Search is a **term-based** information retrieval method that matches a query keyword or phrase against keywords within stored documents. It utilises a retrieval mechanism that matches query tokens precisely against an inverted index of document tokens.



Lexical Search

Information Retrieval

Lexical Search is a **term-based** information retrieval method that matches a query keyword or phrase against keywords within stored documents. It utilises a retrieval mechanism that matches query tokens precisely against an inverted index of document tokens.





Lexical Search

Information Retrieval

Lexical Search is a **term-based** information retrieval method that matches a query keyword or phrase against keywords within stored documents. It utilises a retrieval mechanism that matches query tokens precisely against an inverted index of document tokens.

Limitations

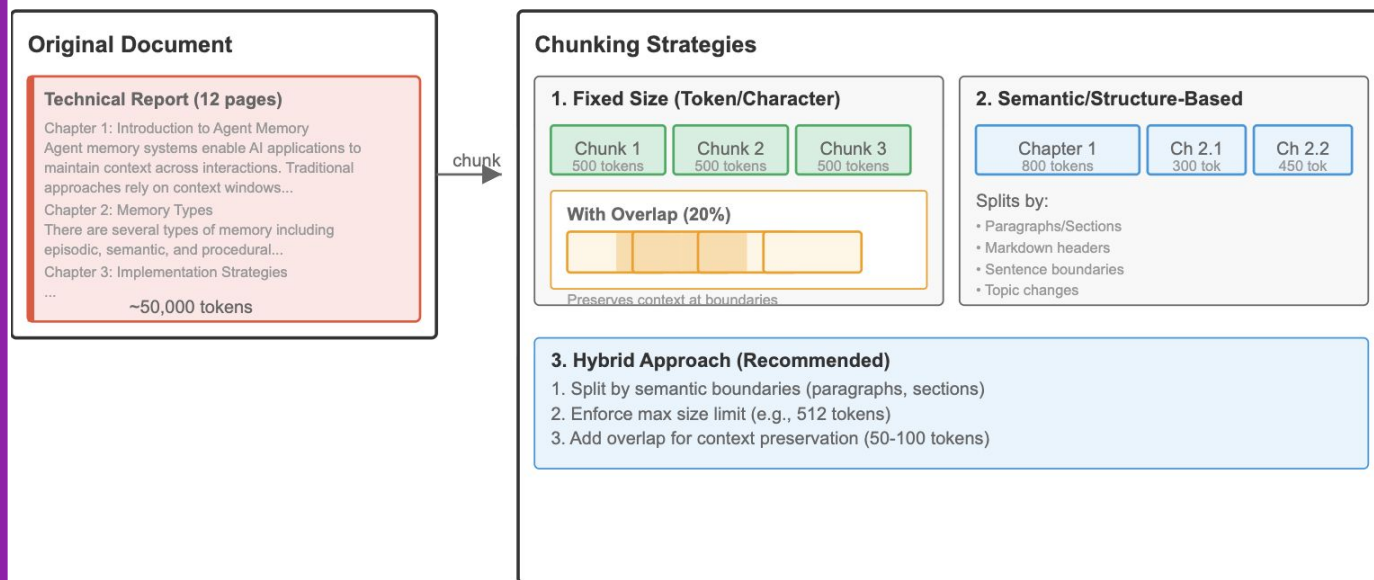
- No Deep Semantics: It doesn't understand meaning. A query for "security breach" won't surface documents about "data leak" unless those exact terms appear.
- Vocabulary Mismatch: It won't match "car" to "automobile" or "password reset" to "forgotten credentials" unless you explicitly define synonym lists

Chunking

Information Retrieval

Chunking is the process of breaking large documents or text into smaller, more manageable pieces (“chunks”) before encoding them into embeddings or indexing for search.

Document Chunking Process in RAG Pipeline



Chunking

Information Retrieval

Chunked Output with Metadata

Chunk #1

```
content: "Agent memory systems..."
doc_id: "tech_report_001"
chunk_id: "chunk_001"
position: 0
tokens: 512
overlap_prev: null
overlap_next: 100
```

Chunk #2

```
content: "...context windows. Memory..."
doc_id: "tech_report_001"
chunk_id: "chunk_002"
position: 1
tokens: 512
overlap_prev: 100
overlap_next: 100
```

Chunk #3

```
content: "...types including episodic..."
doc_id: "tech_report_001"
chunk_id: "chunk_003"
position: 2
tokens: 512
overlap_prev: 100
overlap_next: 100
```

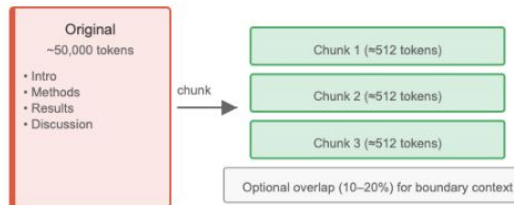
Chunking Stats

Original: 50,000 tokens
Chunks created: 98
Avg chunk size: 512 tokens
Overlap: 100 tokens
Strategy: Hybrid
✓ Ready for embedding

Chunking Strategies

Information Retrieval

1) Fixed Size (Token/Character)



Strengths

- Simple, fast, and predictable at scale
- Works well with streaming/log-like data
- Easy to tune (size, overlap)

Limitations

- Cuts across semantics; topic bleed at boundaries
- May dilute relevance if too large; loses context if too small
- Requires overlap to preserve continuity

When to use

- Uniform text, logs, code, transcripts
- High-throughput indexing where simplicity matters
- Baseline chunking before adding semantics/rerankers

Typical params: 256–1024 tokens (=512 common)

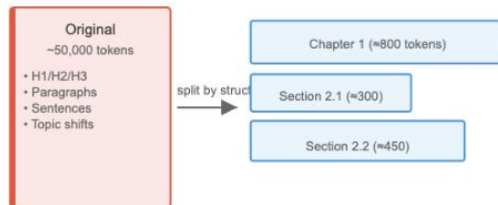
Overlap: 10–20% of chunk size

Splitters: token/character/recursive

Respect code/markdown blocks if possible

Evaluate with retrieval metrics (P@K, nDCG)

2) Semantic / Structure-Based



Strengths

- Respects natural boundaries (chapters/paragraphs)
- Higher coherence; better semantic recall
- Often fewer reranker corrections needed

Limitations

- Chunk sizes vary; may exceed budget without guardrails
- Requires NLP parsing/heuristics; more compute
- Inconsistent density can affect ranking

When to use

- Prose-heavy docs, manuals, research papers
- Markdown/HTML with headers and sections
- When coherence and topicality matter most

Guardrails: enforce a max size (e.g., 512–800 tokens)

Add small overlaps (50–100 tokens) for continuity

Fallback to fixed-size if no clear boundaries

Track retrieval quality; adjust splitters accordingly

1. Fit Model Constraints
2. Improve Retrieval Granularity
3. Enhance Semantic Coherence
4. Support Overlap & Redundancy

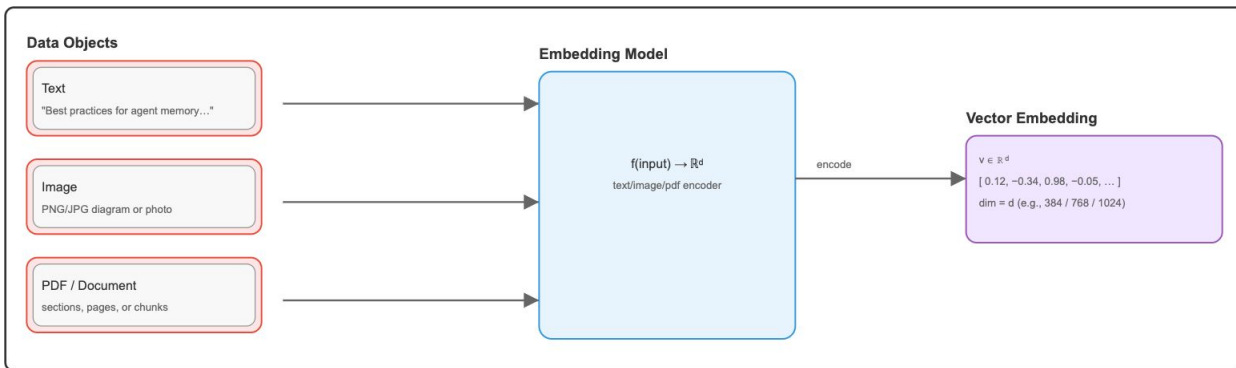
Embedding Models

Information Retrieval

An **embedding model** is a neural encoder that maps inputs (text, images, etc.) to dense, fixed-length vectors such that semantic similarity becomes geometric closeness under a metric (cosine, dot, or L2); trained (often contrastively) to pull related items together and push unrelated ones apart.

It powers tasks like search, clustering, recommendations, and RAG by encoding queries and documents in the same space.

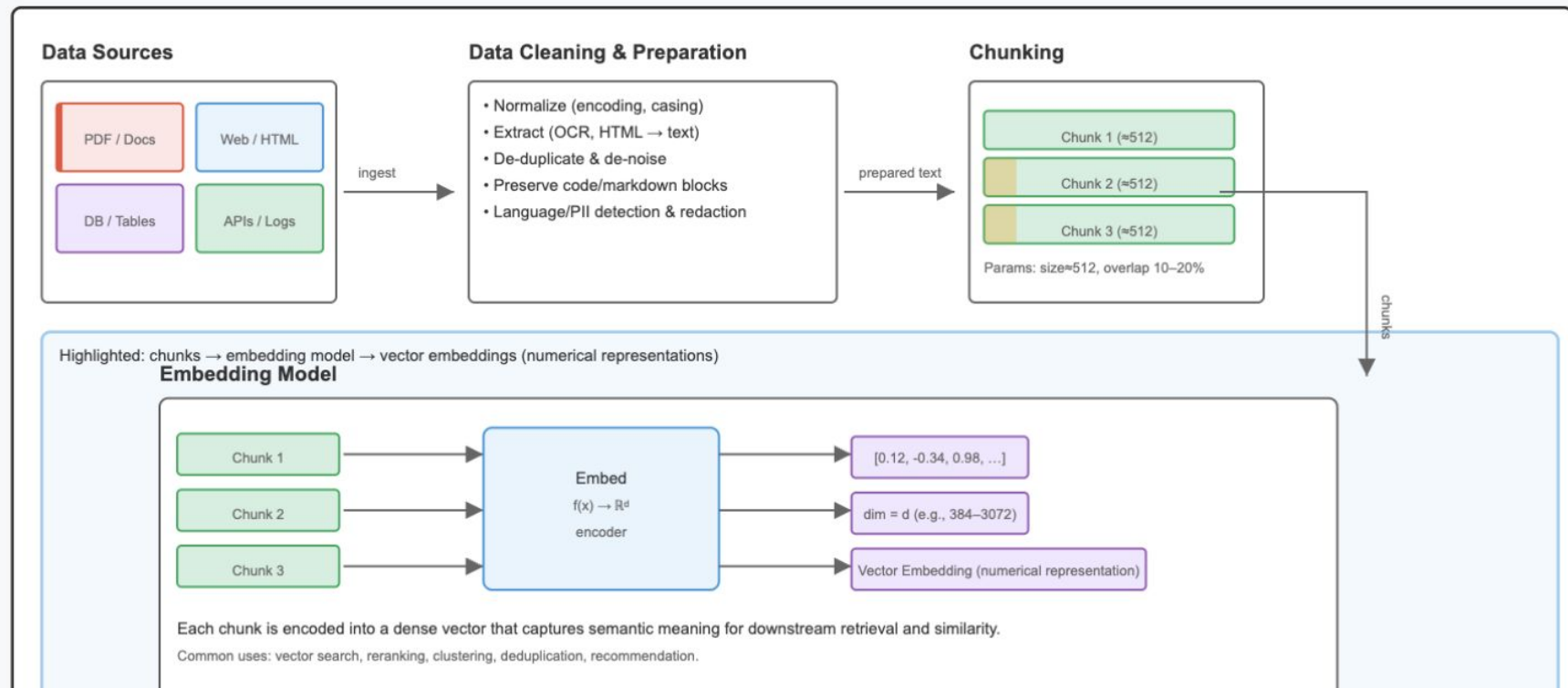
Embedding: Data Object $\rightarrow$ Embedding Model $\rightarrow$ Vector in $\mathbb{R}^d$



Embedding Models

Information Retrieval

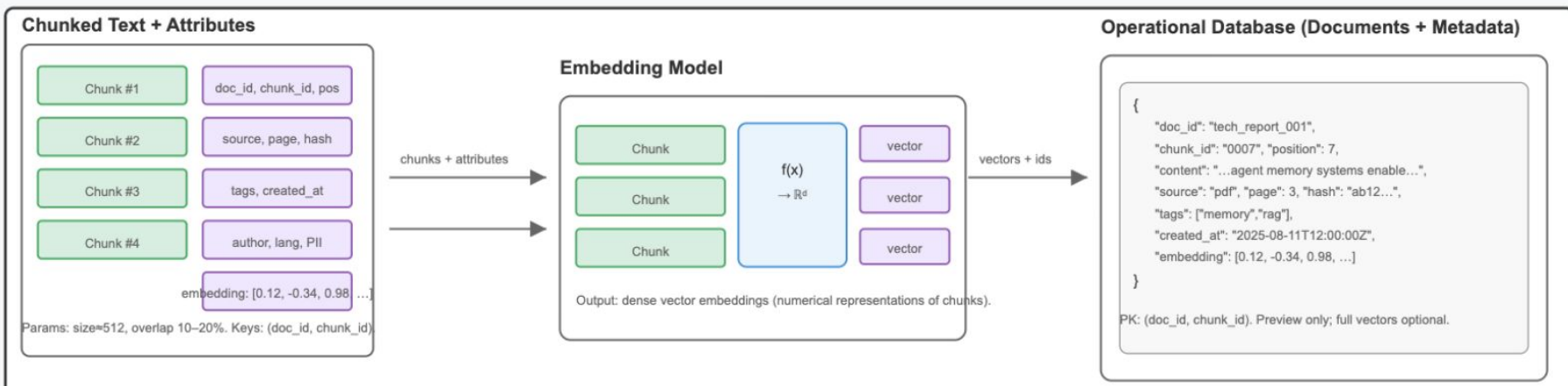
RAG Ingestion Flow: Sources → Clean & Prep → Chunking → Embedding → Vector Embeddings



Embedding Models

Information Retrieval

From Chunks to Storage: Embeddings + Metadata → Operational DB



Why Embedding Models

Information Retrieval

Context Engineering is the systematic practice of **dynamically constructing and managing the information** fed into a large language models context window to optimize inference outcomes.

1. Semantic Search (Retrieval Augmented Generation)
2. Feature Extraction (Classifiers, clustering)

List of Common Embedding Models

Embedding Models

OpenAI — text-embedding-3-small, text-embedding-3-large (current); legacy: text-embedding-ada-002. ([OpenAI Platform](#), [OpenAI](#))

Cohere — embed-english-v3.0, embed-english-light-v3.0, embed-multilingual-v3.0, embed-multilingual-light-v3.0. ([docs.cohere.com](#))

Voyage AI — voyage-3.5, voyage-3.5-lite, voyage-3-large, voyage-code-3, voyage-context-3, voyage-finance-2, voyage-law-2 (plus older: voyage-3, voyage-3-lite). ([Voyage AI](#), [Voyage AI](#))

(Vendors add/retire models often—always double-check the docs before choosing.)

Vector Search

Information Retrieval

Vector Search (aka semantic search) is a retrieval method that matches query vectors and documents **based on their positions in a high-dimensional** embedding space, capturing semantic similarity rather than exact term overlap.

High-Dimensional Vector Space (visualized via 2D projection)



Distance Functions

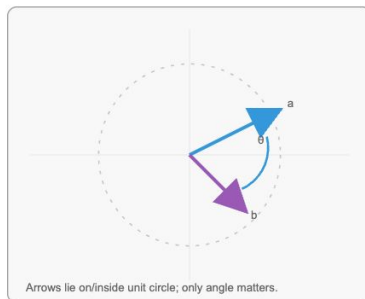
Vector Search

A distance function **quantifies how far apart two objects are in a given space**, turning intuition about similarity into numbers.

It drives tasks like nearest-neighbor search, clustering, and retrieval by ranking candidates—where the chosen distance (e.g., cosine, dot product, Euclidean) defines what “similar” means for your application.

Intuitive Comparison: Cosine • Dot Product • Euclidean

1) Cosine Similarity (angle only)



What it measures

- How aligned two vectors are (the angle between them).
- Ignores overall length; focuses on direction.

Range & behavior

- Range: $[-1, 1]$ (1 = same direction, 0 = orthogonal, -1 = opposite).
- Common when vectors are length-normalized.

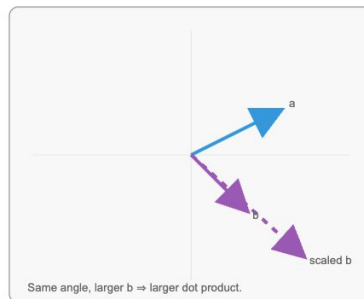
Formula

$$\cos(a, b) = (a \cdot b) / (|a||b|)$$

Intuition

- “Are they pointing the same way?” Topic similarity.

2) Dot Product (alignment × magnitude)



What it measures

- Alignment weighted by lengths: $|a||b|\cos\theta$.
- Sensitive to vector magnitudes (scale).

Range & behavior

- Unbounded (depends on norms of a and b).
- If vectors aren't normalized, bigger = stronger signal.

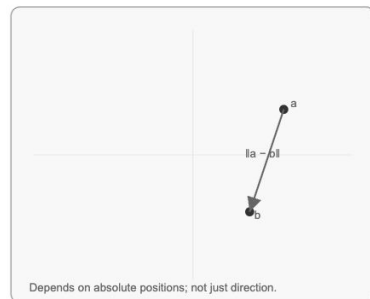
Formula

$$a \cdot b = \sum a_i b_i = |a||b|\cos\theta$$

Intuition

- “How much of b lies along a, scaled by sizes?”

3) Euclidean Distance (straight-line gap)



What it measures

- The straight-line distance between two points in space.
- Sensitive to both direction and magnitude.

Range & behavior

- Non-negative; grows with separation.
- For unit-normalized vectors, lower distance = higher cosine.

Formula

$$|a - b|_2 = \sqrt{\sum (a_i - b_i)^2}$$

Intuition

- “How far apart are the points?”

HNSW (Hierarchical Navigable Small World)

Vector Search

HNSW is an approximate nearest-neighbor index that organizes vectors into a multi-layer small-world graph.

Queries start at a sparse top layer, greedily move toward closer nodes via long-range links, then descend to denser layers to refine results—achieving fast (sub-linear) search with high recall.

Key Parameters:

M: neighbors per node (higher → better recall, more memory).

efConstruction: build-time breadth (higher → better graph, slower builds).

efSearch: query-time breadth (higher → better recall, higher latency).

Distance function: must match your embeddings; changing it means rebuilding.

HNSW (Hierarchical Navigable Small World)

Vector Search

IEEE TRANSACTIONS ON JOURNAL NAME, MANUSCRIPT ID

1

Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs

Yu. A. Malkov, D. A. Yashunin

Abstract — We present a new approach for the approximate K -nearest neighbor search based on navigable small world graphs with controllable hierarchy (Hierarchical NSW, HNSW). The proposed solution is fully graph-based, without any need for additional search structures, which are typically used at the coarse search stage of the most proximity graph techniques. Hierarchical NSW incrementally builds a multi-layer structure consisting from hierarchical set of proximity graphs (layers) for nested subsets of the stored elements. The maximum layer in which an element is present is selected randomly with an exponentially decaying probability distribution. This allows producing graphs similar to the previously studied Navigable Small World (NSW) structures while additionally having the links separated by their characteristic distance scales. Starting search from the upper layer together with utilizing the scale separation boosts the performance compared to NSW and allows a logarithmic complexity scaling. Additional employment of a heuristic for selecting proximity graph neighbors significantly increases performance at high recall and in case of highly clustered data. Performance evaluation has demonstrated that the proposed general metric space search index is able to strongly outperform previous opensource state-of-the-art vector-only approaches. Similarity of the algorithm to the skip list structure allows straightforward balanced distributed implementation.

Index Terms — Graph and tree search strategies, Artificial Intelligence, Information Search and Retrieval, Information Storage and Retrieval, Information Technology and Systems, Search process, Graphs and networks, Data Structures, Nearest neighbor search, Big data, Approximate search, Similarity search

1 INTRODUCTION

Constantly growing amount of the available information resources has led to high demand in scalable and efficient similarity search data structures. One of the generally used approaches for information search is the K -Nearest Neighbor Search (K-NNS). The K-NNS assumes you have a defined distance function between the data elements and aims at finding the K elements from

errors. The quality of an inexact search (the recall) is defined as the ratio between the number of found true nearest neighbors and K . The most popular K-ANNS solutions are based on approximated versions of tree algorithms [6, 7], locality-sensitive hashing (LSH) [8, 9] and product quantization (PQ) [10-17]. Proximity graph K-ANNS algorithms [10, 18-26] have recently gained popu-

HNSW (Hierarchical Navigable Small World)

Vector Search

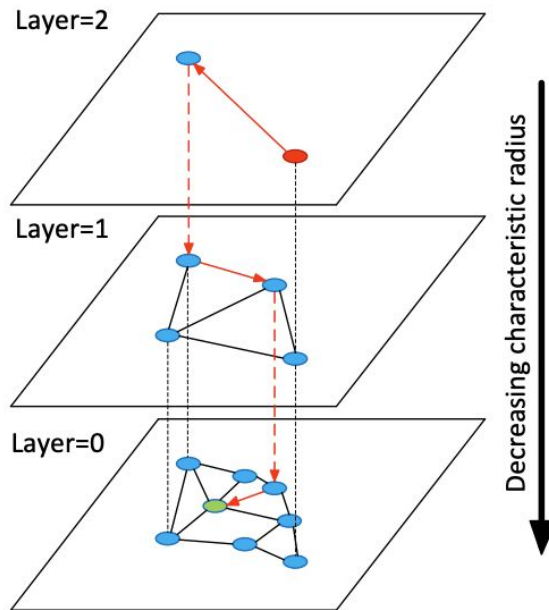


Fig. 1. Illustration of the Hierarchical NSW idea. The search starts from an element from the top layer (shown red). Red arrows show direction of the greedy algorithm from the entry point to the query (shown green).

Vector Search

Information Retrieval

Vector Search (aka semantic search) is a retrieval method that matches query vectors and documents **based on their positions in a high-dimensional** embedding space, capturing semantic similarity rather than exact term overlap.

Other Indexes for Vector Search

IVF (Inverted File) — IVF-Flat / IVF-PQ: Partitions vectors into clusters (via k-means); searches only a few lists, then re-ranks. With **PQ/OPQ** you compress vectors to save RAM. Great latency/scale trade-off for large datasets.

Annoy (Random Projection Trees): Builds a forest of random hyperplane trees; very fast queries and tiny build deps, but best for **static** data (updates require rebuild). Good recall/latency for many apps.

Limitations

- Resource-Intensive: Generating embeddings at ingest and query time adds compute cost.
- Index Complexity: ANN indexes require tuning (e.g. choosing M, efConstruction for HNSW).
- Explainability: Semantic scores are less interpretable than exact term matches.

Hybrid Search

Information Retrieval

Hybrid Search is an information retrieval approach that combines lexical (term-based) search and vector (semantic) search to leverage the strengths of both methods.

User Query

"Find documentation about memory management in AI agents"

Intent: Technical docs on AI memory systems | Keywords: memory, management, agents

Lexical Search Path

Token Analysis

Tokens: ["memory", "management", "AI", "agents"]
Stemmed: ["memori", "manag", "ai", "agent"]

BM25 Scoring

- Term frequency (TF)
- Inverse doc frequency (IDF)
- Field boosts (title > body)
- Exact phrase matching

Lexical Results (Exact Matches)

1. "Memory Management Strategies" (0.92)
2. "Agent Memory Best Practices" (0.85)
- ⋯ Missing: "Cognitive Architecture Guide"

Strengths: Exact terms, precision

Vector Search Path

Query Embedding

Model: text-embedding-3
Dimensions: 1536
Vector: [0.23, -0.67, 0.89...]
Captures semantic meaning

Cosine Similarity

- k-NN search (k=10)
- ANN index (HNSW)
- Semantic matching
- Cross-lingual capable

Vector Results (Semantic Matches)

1. "Cognitive Architecture Guide" (0.89)
2. "Neural Memory Networks" (0.86)
3. "Information Retention Systems" (0.83)

Strengths: Synonyms, concepts, context

Hybrid Fusion & Reranking

Fusion Methods

1. Reciprocal Rank Fusion (RRF)
 $score = \sum (1 / (k + rank))$
2. Weighted Linear Combination
 $score = \alpha \cdot lexical + \beta \cdot vector$
typical: $\alpha=0.3$, $\beta=0.7$
3. ML Reranker (optional)

Hybrid Results (Best of Both)

1. "Memory Management Strategies" ⚡📊
 2. "Cognitive Architecture Guide" 🧠
 3. "Agent Memory Best Practices" ⚡
- ⚡=lexical match | 🧠=semantic match

Hybrid Benefits

- ✓ Captures exact matches
 - ✓ Understands synonyms
 - ✓ Handles typos/variations
 - ✓ Better recall & precision
 - ✓ Robust to query types
- ~40% better than either alone

Reciprocal Rank Fusion (RRF)

Hybrid Search

Reciprocal Rank Fusion (RRF)
is a simple way to merge
multiple ranked result lists
(e.g., vector and lexical) into
one.

$$\text{RRF}(d) = \sum_L \frac{1}{k + \text{rank}_L(d)}$$

Reciprocal Rank Fusion (RRF)

Hybrid Search

Ranked List A — Vector (ANN)

- 1) A = tech_report_001#007
- 2) B = guide_12#014
- 3) C = blog_mem#221
- 4) D = whitepaper_X#031
- 5) E = howto#088

Ranked List B — Lexical (BM25)

- 1) B = guide_12#014
- 2) F = faq_77#009
- 3) A = tech_report_001#007
- 4) G = paper_Z#312
- 5) C = blog_mem#221

RRF Formula

$$\text{score}(d) = \sum_{\text{lists } L} \frac{1}{(k + \text{rank}_L(d))}$$

- If d is missing in a list $\Rightarrow$ contribution 0
- Typical k = 60 (used below)

Example (k = 60)

Doc	rank A	rank B	RRF components	Score
B	2 $\rightarrow$ 1/62	1 $\rightarrow$ 1/61	1/62 + 1/61	0.03252
A	1 $\rightarrow$ 1/61	3 $\rightarrow$ 1/63	1/61 + 1/63	0.03227
C	3 $\rightarrow$ 1/63	5 $\rightarrow$ 1/65	1/63 + 1/65	0.03126
F	$\rightarrow$ 0	2 $\rightarrow$ 1/62	0 + 1/62	0.01613
D	4 $\rightarrow$ 1/64	$\rightarrow$ 0	1/64 + 0	0.01563
G	$\rightarrow$ 0	4 $\rightarrow$ 1/64	0 + 1/64	0.01563
E	5 $\rightarrow$ 1/65	$\rightarrow$ 0	1/65 + 0	0.01538

Fused Top-K (by RRF score)

- 1) B = guide_12#014 • 0.03252
- 2) A = tech_report_001#007 • 0.03227
- 3) C = blog_mem#221 • 0.03126
- 4) F = faq_77#009 • 0.01613
- 5) D = whitepaper_X#031 • 0.01563

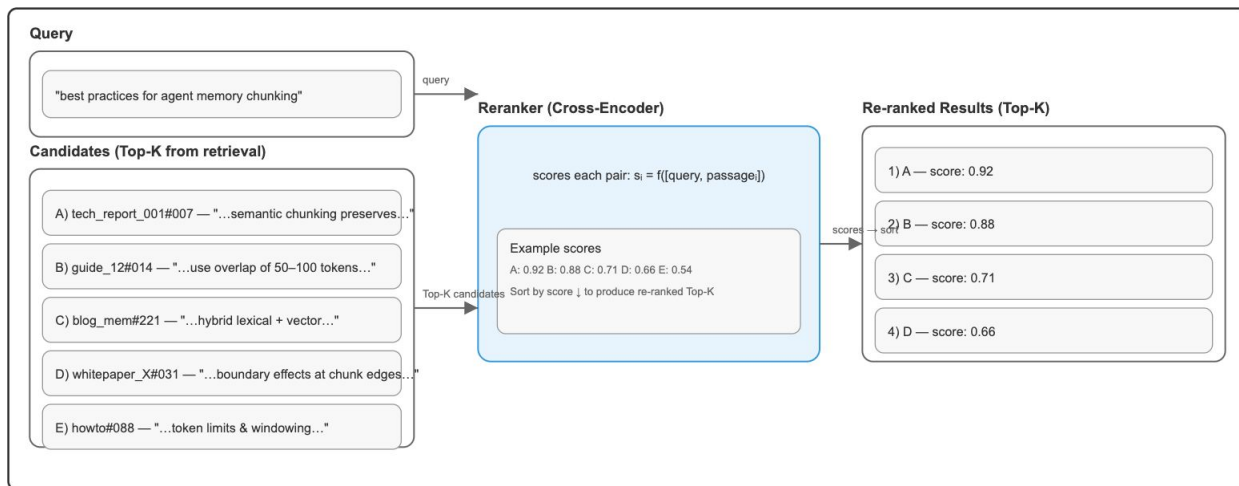
Ties broken by secondary rule (e.g., source preference, recency).

Re-Rankers

Information Retrieval

Re-Rankers are models used in multi-stage retrieval systems to improve the final ordering of previously retrieved candidate documents by applying a more precise scoring after an initial “fast” retrieval.

Simple Reranker (Cross-Encoder): Query + Candidates → Scores → Re-ranked Results

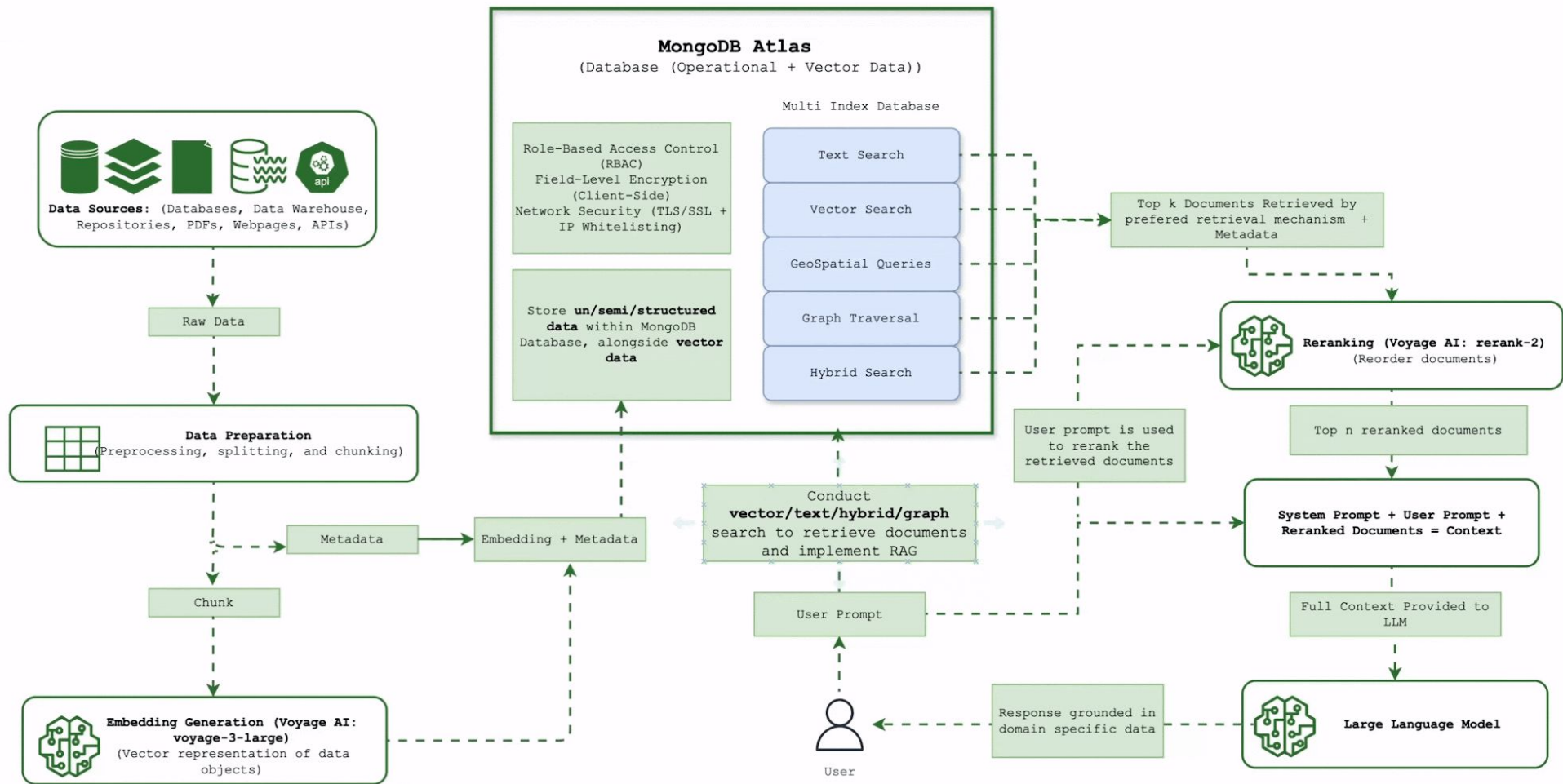


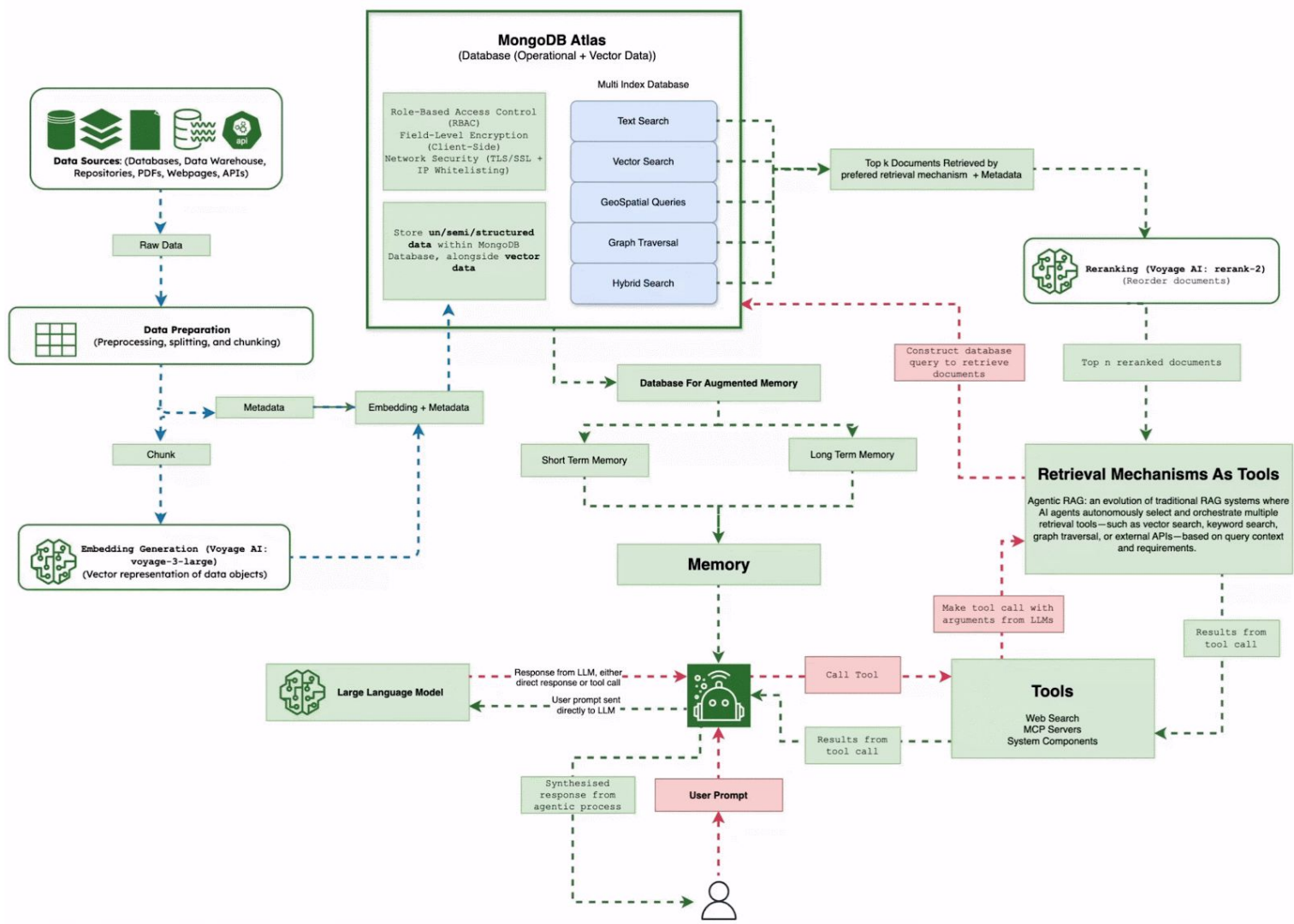
Why Re-Rankers

Information Retrieval

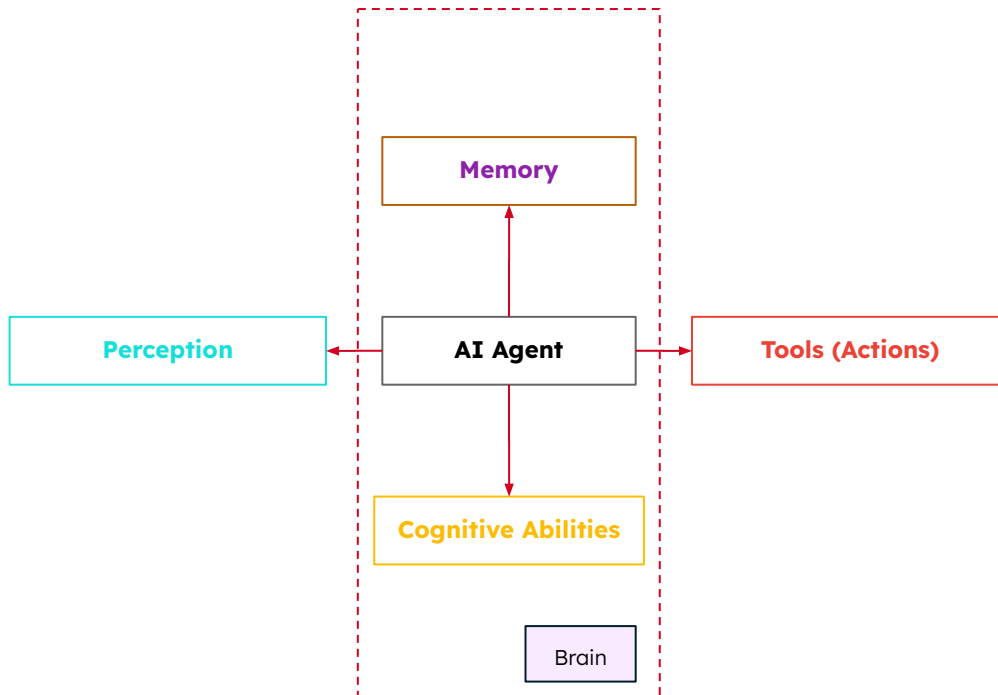
Re-Rankers are models used in multi-stage retrieval systems to improve the final ordering of previously retrieved candidate documents by applying a more precise scoring after an initial “fast” retrieval.

1. Efficiency vs. Quality Trade-off
2. Second-Stage Refinement

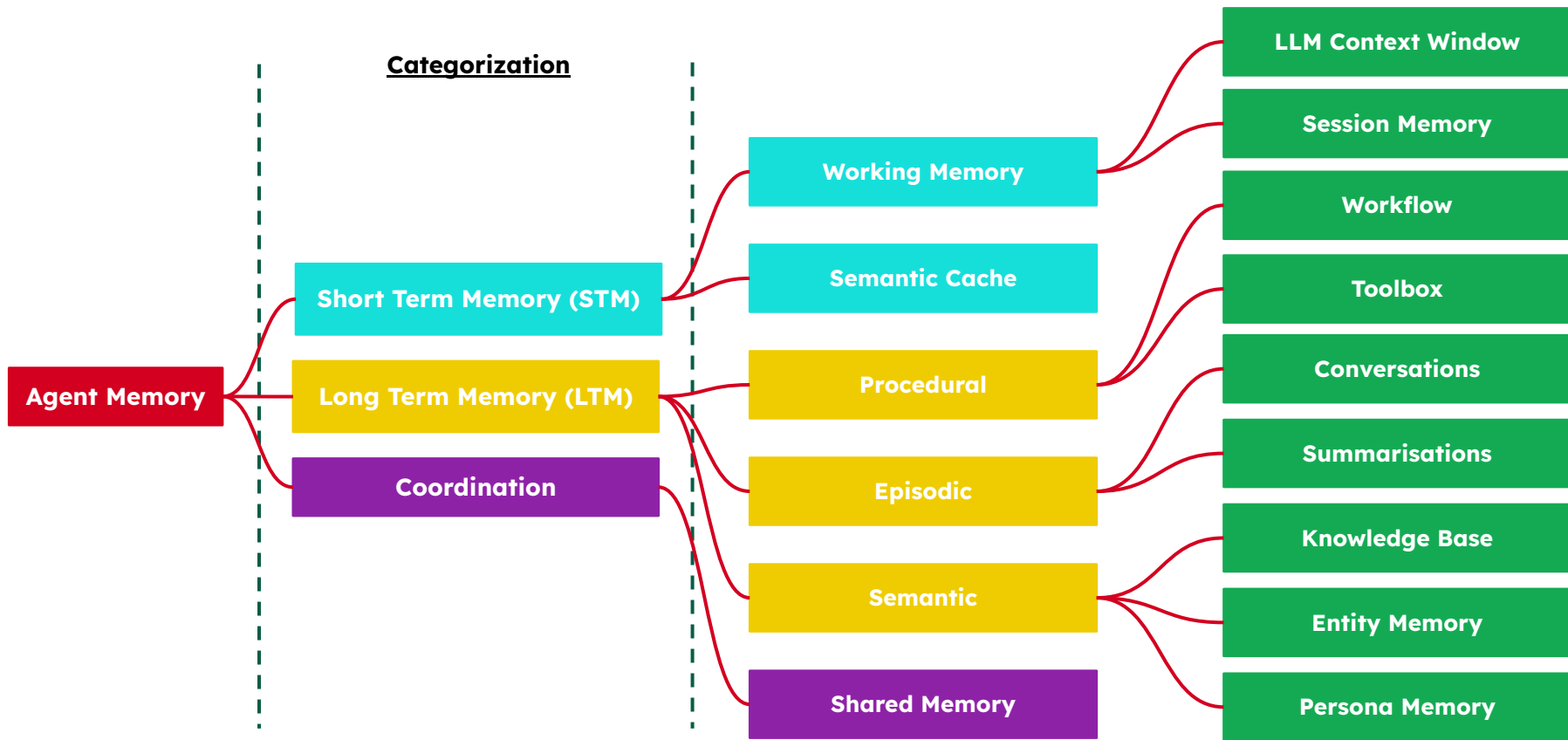




Recap: AI Agent



Recap: Agent Memory Types



Recap: Memory Engineering

Agent Memory Engineering is the discipline focused on **designing and optimizing persistent memory** substrates, **retrieval mechanisms**, and **memory lifecycle management** strategies that enable AI agents to accumulate knowledge, maintain contextual awareness, and adapt behavior through sophisticated cognitive architectures.

Poll Question

Question: Which retrieval method combines keyword matching with semantic similarity?

- A) Lexical Search
- B) Vector Search
- C) Hybrid Search
- D) Procedural Memory Search



Poll Question

Question: Which retrieval method do you currently use or plan to use most?

- A) Lexical Search (BM25, inverted index)
- B) Vector Search (semantic embeddings)
- C) Hybrid Search
- D) Not sure yet



Q&A





Break

Module 3: AI and Agent Memory

- Defining Memory provider, memory layers
- What is a Memory Provider
- What is a Memory Layer
- Handling Memory with LangMem, LangChain and LangGraph
- Handling Memory with LlamaIndex
- Handling Memory with Memory Bank
- Handling Memory with MemGPT
- Handling Memory with Mem0
- Handling Memory with Zep



Steps For GitHub Repo

Prerequisites

- Python 3.8+ installed
- Git installed
- Jupyter Notebook or JupyterLab installed
- MongoDB account and cluster
- Voyage AI API key

Step 1: Clone the Repository

```
git clone https://github.com/RichmondAlake/agent_memory_course  
cd agent_memory_course
```

Step 2: Set Up Python Environment

Create and activate a virtual environment:

```
python -m venv venv  
source venv/bin/activate # On Windows: venv\Scripts\activate
```

Step 3: Install Dependencies

```
pip install langmem langgraph langchain-voyageai jupyter
```

Steps For GitHub Repo

Step 4: Set Up Required Services

MongoDB Setup

1. Create a MongoDB Atlas account at <https://www.mongodb.com/atlas>
2. Create a new cluster
3. Get your MongoDB connection URI from the cluster dashboard
4. Ensure your IP is whitelisted in the Network Access settings

Voyage AI Setup

1. Sign up at <https://www.voyageai.com/>
2. Get your API key from the dashboard

Step 5: Launch Jupyter

jupyter notebook
or
jupyter lab

Step 6: Run the Notebooks

1. Navigate to the langmem/ folder
2. Open procedural_memory.ipynb
3. When prompted, enter your:
 - MongoDB URI
 - Voyage AI API key

Resources

1. [“LangMem SDK for agent long-term memory”](#) (blog post, Feb 18, 2025) – Introduces the launch of the LangMem SDK, explaining its purpose, integration options, memory types, and how it enhances agent intelligence.
2. Developer conceptual guide: [“Long-term Memory in LLM Applications”](#) – Defines memory types (semantic, episodic, procedural) and how LangMem models them.
3. DeepLearning.AI short course: [“Long-Term Agentic Memory with LangGraph”](#) – A free, instructor-led video course (by Harrison Chase) that demonstrates constructing an agent with semantic, episodic, and procedural memory (e.g., email agent example with code) over several lessons.
4. ArXiv Research Paper: [“Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory”](#) (Apr 2025) – A formal research paper introducing Mem0 and its graph-based memory architecture. It outperforms existing memory methods on reasoning benchmarks (like LOCOMO), offering efficiency and improved performance.
5. Research Paper: [“MemGPT: Towards LLMs as Operating Systems”](#) (arXiv:2310.08560) – Introduces “virtual context management” to overcome LLM context limits, inspired by operating system memory hierarchies. Demonstrates improvements in document analysis and multi-session chat.
6. [Vertex AI Memory Bank](#) (Public Preview by Google) – Managed service allowing agents to extract, store, and retrieve memories across sessions using Gemini and semantic search. Integrated with Google’s Agent Development Kit (ADK) and compatible with frameworks like LangGraph and CrewAI

Poll Question

Question: Which of these is not a core AI memory type covered in this course?

- A) Episodic Memory
- B) Procedural Memory
- C) Semantic Memory
- D) Predictive Memory



Poll Question

Question: Which library or framework are you most interested in experimenting with for memory management?

- A) LangMem / LangGraph
- B) MemGPT
- C) Mem0
- D) MemoryBank
- E) Zep
- F) LlamaIndex
- G) Other / Not sure yet



Q&A





Break

Module 4: Memory Management Implementation

- Overview of MemoRizz
- Memory Type Implementation
- Memory Type Utilization in AI Agents



Steps For GitHub Repo

Prerequisites

- Python 3.8+ installed
- Git installed
- Jupyter Notebook or JupyterLab installed
- MongoDB account and cluster
- Voyage AI API key

Step 1: Clone the Repository

```
git clone https://github.com/RichmondAlake/agent_memory_course  
cd agent_memory_course
```

Step 2: Set Up Python Environment

Create and activate a virtual environment:

```
python -m venv venv  
source venv/bin/activate # On Windows: venv\Scripts\activate
```

Step 3: Install Dependencies

```
pip install langmem langgraph langchain-voyageai jupyter
```

Steps For GitHub Repo

Step 4: Set Up Required Services

MongoDB Setup

1. Create a MongoDB Atlas account at <https://www.mongodb.com/atlas>
2. Create a new cluster
3. Get your MongoDB connection URI from the cluster dashboard
4. Ensure your IP is whitelisted in the Network Access settings

Voyage AI Setup

1. Sign up at <https://www.voyageai.com/>
2. Get your API key from the dashboard

Step 5: Launch Jupyter

jupyter notebook
or
jupyter lab

Step 6: Run the Notebooks

1. Navigate to the langmem/ folder
2. Open procedural_memory.ipynb
3. When prompted, enter your:
 - MongoDB URI
 - Voyage AI API key

Poll Question

Question: After this course, where do you see yourself applying memory engineering techniques first?

- A) Customer support bots
- B) Research assistants / knowledge workers
- C) Multi-agent systems
- D) Other AI applications



Resources

1. [Anthropic – Multi-Agent Research System](#): Explains how complex, production-ready AI agents manage information. Notably avoids the term “*database*”, framing storage as “*memory*”—a shift in language important for database providers.
2. [Don't Just Build Agents, Build Memory-Augmented Agents](#) – Richmond Alake: Makes the case for memory-augmented architectures as the default for agent development. Analyzes Cognition and Anthropic examples; introduces AI memory engineering as key to reliability.
3. [Stanford CS229 – Building Large Language Models \(LLMs\)](#): A clear, technically solid overview of LLM architectures and the GenAI space. Balances theory and application—ideal foundation for understanding agent capabilities.
4. Survey: [Memory Mechanisms in LLM Agents](#): Detailed academic review of how memory is implemented in LLM-based agents. Covers architectures, techniques, and operational patterns.
5. Survey: [From Human Memory to AI Memory](#): Draws parallels between human cognitive memory and AI memory designs. Offers conceptual and technical framing for building more human-like agent memory.

Takeaway

1. More reading list
 - a. [Zep: A Temporal Knowledge Graph Architecture for Agent Memory](#)
 - b. [LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory](#)
2. Reach out to me
 - a. LinkedIn: <https://www.linkedin.com/in/richmondalake/>
 - b. Twitter: <https://x.com/richmondalake>
3. [Carry on the discussion and join the Agent Memory Discord Channel:](#)
<https://discord.gg/kGs5hKy8n8>

Q&A



The image features the O'Reilly logo in white, centered on a blue background. The logo consists of the word "O'REILLY" in a bold, sans-serif font, followed by a registered trademark symbol (®). To the left of the text, there are two large, overlapping, semi-transparent blue circles that create a sense of depth and movement. The background is a solid blue color with a slight gradient, transitioning from a darker blue on the left to a lighter blue on the right.

O'REILLY®